studeersnel

STM32F103C8T6

Điều khiển nhúng (Trường Đại học Bách khoa Hà Nội)

Scan to open on Studeersnel

Studocu is not sponsored or endorsed by any college or university
Downloaded by Du Di?p (diepdu2005@gmail.com)

STM32F103C8T6

Contents

A. Giới thiệu tổng quan 4

B. Giới thiệu 7

I. Tổng quan 7

1. **Mục tiêu quan trọng** 7
2. **Hỗ trợ** 7

II. Ứng dụng của vi điều khiển, sự phổ biến của dòng vi xử lý ARM Cortex M 7

1. **Sự phổ biến** 7

III. Các khái niệm CPU CORE, CPU và MCU 8

1. **Khái niệm Processor Core và Processor** 8
2. **Khái niệm Processor và Microcontroller.** 9

C. Phần cứng 10

I. Phân tích và lựa chọn board 10

1. **Tiêu chí lựa chọn** 10

II. Chi tiết Board Discovery và Nucleo STM32F411 11

1. **Nguồn** 11
2. **Led** 11
3. **Sơ đồ nguyên lý** 11
4. **Button: RESET** 11
5. **Header** 12
6. **Thạch anh** 12

D. STM32CubeIDE 12

I. Tổng quan và cài đặt 12

1. **Tính năng** 12
2. **Cài đặt** 12
3. **Tạo project** 12

IV. Cấu trúc của Project 13

1. **Bố cục của chương trình C** 13
2. **Thao tác điều khiển** 13
3. **Driver** 14

E. Thư viện và chương trình mẫu 14

Downloaded by Du Di?p (diepdu2005@gmail.com)

I. Tìm hiểu về CMSIS 14
II. Thư viện HAL 15
III. Chương trình mẫu 15
IV. Debug 15
1. Các tính năng Debug 15
2. Điều khiển quá trình thực thi của chương trình. 16
F. Kiến trúc VXL ARM 16
I. Chế độ hoạt động 16
II. Các thanh ghi 16
III. Hệ thống Bus 16
IV. Trình tự khởi động 18
1. Chương trình trong bộ nhớ 18
2. Quá trình khởi động của vi xử lý 19
G. Tổ chức bộ nhớ 20
I. Chức năng bộ nhớ 20
II. Lưu trữ và chuyển dữ liệu Alidned, Unaligned 22
III. Bit – Banding 23
H. System Exceptions and Interrupt 25
I. System Exception 25
II. Interrupt 26
1. Tổng quan về thiết kế ngắt 26
2. Vấn đề 27
II. NVIC 27
1. Các thanh ghi 27
III. Interrupt Priority 28
IV. Pending Interrupt 29
V. Quy trình vào ra ngắt của VXL 29
VI. Stacking và Unstacking khi xử lý ngắt 29
J. Nguồn cấp và bộ dao động 30
I. Khối nguồn 30
II. Chế độ tiết kiệm năng lượng 31
1. Các tác nhân ảnh hưởng năng lượng 31
2. Các chế độ hoạt động (M4) 32

Downloaded by Du Di?p (diepdu2005@gmail.com)

III. Clock tree 33
1. **Nguồn cấp Clock** 34
2. **Clock tree** 34
3. **Cấu hình Clock Tree.** 34
K. Thiết kế luồng xử lý của chương trình. 35
I. Polling 35
II. Interrupt 35
III. DMA 36
L. Lập trình ngoại vi GPIO 37
I. Cấu trúc và chức năng của khối GPIO 37
II. GPIO Output với LED 39
III. GPIO Input với Button 40
M. Lập trình ngắt ngoài EXTI. 40
I. Cấu trúc và chức năng của khối ngắt ngoài EXTI. 40
II. EXTI với Button 42
N. Giao tiếp UART 42
I. Cấu trúc và chức năng khối USART. 42
1. **Kết nối phần cứng** 42
2. **Khung truyền dữ liệu.** 43
3. **Tốc độ Baud** 43
II. Truyền dữ liệu nhiều định dạng qua UART. 43
III. Nhận dữ liệu UART sử dụng ngắt nhận RXNEIE. 44

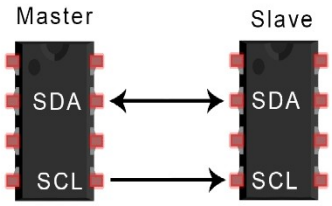
Downloaded by Du Di?p (diepdu2005@gmail.com)

A. Giới thiệu tổng quan

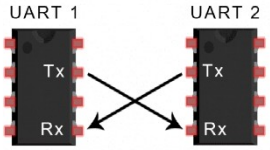
- STM32F103C8T6 là vi điều khiển 32 bit, thuộc họ F1.
- Lõi ARM COTEX M3
- Tốc độ tối đa: 72Mhz
- Bộ nhớ:
    - 64KB bộ nhớ Flash
        - Là bộ nhớ chương trình (Firmware).
        - Không bị mất khi mất nguồn.
        - Chỉ đọc.
    - 20 KB SRAM
        - Bộ nhớ dữ liệu.
        - Mất dữ liệu khi mất nguồn.
        - Không cần làm mới dữ liệu (như DRAM).
- Clock, Reset và quản lý nguồn
    - Điện áp: 2 – 3.6 V
    - Thạch anh ngoài: 4MHz – 20MHz
    - Thạch anh nội dùng dao động RC ở mode 8MHz hoặc 40MHz
        - Dao động RC: Mạch gồm R và C.
            - Tạo tín hiệu dao động dựa trên thời gian tụ điện sạc và xả.
- Chế độ điện áp thấp:
    - Có các mode: ngủ, ngừng hoạt động hoặc hoạt động ở chế độ chờ.
    - Cấp nguồn ở chân Vbat bằng pin ngoài để dùng bộ RTC và sử dụng dữ liệu được lưu trữ khi mất nguồn cấp chính.
        - Bộ RTC (Real Time Clock)
            - Có nguồn cấp riêng là Vbat để duy trì khi mất nguồn cấp chính.
            - Khi mất nguồn cấp chính, bộ RTC duy trì đồng hồ thời gian thực và lưu trữ dữ liệu.
- ADC: Có 2 bộ 12 bit với 9 kênh cho mỗi bộ.
    - Khoảng chuyển đổi: 0 – 3.6V.
    - Có chế độ lấy mẫu 1 kênh hoặc nhiều kênh.
- DMA:
    - 7 kênh DMA.
    - Có hỗ trợ DMA cho ADC, UART, I2C, SPI.

Downloaded by Du Di?p (diepdu2005@gmail.com)

- Timer: 7 bộ
    - 3 Timer 16 bit hỗ trợ các mode Input Capture/ Output Compare/ PWM.
    - 1 Timer 16 bit hỗ trợ điều khiển động cơ với các mode bảo vệ ngắt Input, dead-time.
    - 2 Watchdog Timer để bảo vệ và kiểm tra lỗi.
    - 1 Systick Timer 24 bit đếm xuống cho hàm Delay.
- Kênh giao tiếp: 9 kênh
    - 2 bộ I2C.



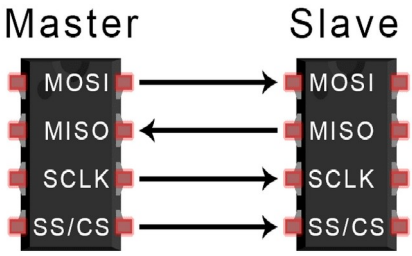
        - Kết hợp ưu điểm của UART và SPI, là truyền tin đồng bộ.
        - Có thể có 1 Master hoặc nhiều Master.
        - Ưu điểm:
            - Chỉ sử dụng 2 dây.
            - Hỗ trợ nhiều Master.
            - Bit ACK/NACK xác nhận mỗi khung được chuyển thành công.
            - Phần cứng ít phức tạp hơn so với UART.
        - Nhược điểm:
            - Tốc độ truyền chậm hơn SPI.
            - Bị giới hạn bởi khung truyền (8 bit).
            - Cần phần cứng phức tạp hơn SPI.
    - 3 bộ USART.



        - Bộ phát thu không đồng bộ, truyền tin nối tiếp.
        - Ưu điểm:
            - Chỉ sử dụng 2 dây.
            - Không cần tín hiệu Clock.
            - Có một bit chẵn lẻ để cho phép kiểm tra lỗi.

Downloaded by Du Di?p (diepdu2005@gmail.com)

        - Cấu trúc gói dữ liệu cho thể thay đổi được nếu 2 bên cho phép thiết lập.
    - Nhược điểm:
        - Kích thước khung dữ liệu bị giới hạn (Tối đa 9 bit).
        - Không hộ trợ nhiều Slave hay nhiều Master.
        - Tốc độ truyền mỗi UART phải nằm trong khoảng 10 % của nhau.
- 2 SPI.



    - Truyền dữ liệu không bị gián đoạn (Không bị giới hạn số lượng bit như UART hay I2C), truyền đồng bộ.
    - Có quan hệ Master – Slave.
    - Ưu điểm:
        - Không có bit bắt đầu và dừng.
        - Không có hệ thống định địa chỉ slave phức tạp như I2C.
        - Tốc độ nhanh x2 I2C.
        - Có thể gửi và nhận cùng một lúc (2 chân MISO và MOSI riêng biệt).
    - Nhược điểm:
        - Sử dụng 4 dây (I2C và UART dùng 2 dây).
        - Không xác nhận dữ liệu đã được nhận thành công (I2C có).
        - Không có hình thức kiểm tra lỗi như bit chẵn lẻ trong UART.
        - Chỉ có 1 Master duy nhất.
- 1 CAN
- USB 2.0 full-speed interface

- Kiểm tra lỗi CRC và 96-bit ID.

**** Kit STM32F103C8T6

Downloaded by Du Di?p (diepdu2005@gmail.com)

- Điện áp cấp 5 VDC qua cổng Micro USB được chuyển đổi thành 3.3 VDC qua IC nguồn và cấp cho Vi điều khiển chính.
- Tích hợp sẵn thạch anh 8MHz.
- Tích hợp sẵn thạch anh 32KHz cho các ứng dụng RTC.
- Ra chân đầy đủ tất cả các GPIO và giao tiếp: CAN, I2C, SPI, UART, USB, …
- Tích hợp LED trạng thái nguồn, LED PC13, Nút Reset.

B. Giới thiệu

I. Tổng quan

1. Mục tiêu quan trọng

- Các kiến thức về kiến trúc Vi xử lý Arm Cortex M, bản đồ không gian bộ nhớ và giao diện Bus.
- Hiểu và quản lý được các ngắt trong một hệ thống, thiết kế được độ ưu tiên khi các ngắt lồng nhau.
- Biết cách khai thác, sử dụng các thư viện dành cho dòng Vi điều khiển STM32.
- Kỹ năng Debug trong quá trình phát triển phần mềm cho hệ thống nhúng với ngôn ngữ lập trình C sử dụn STM32CubeIDE và Keil C IDE.
- Sử dụng được các ngoại vi thông dụng trên dòng Vi điều khiển STM32: GPIO, EXTI, UART.

2. Hỗ trợ

- Trang facebook:
    - MCU Learning
    - TAPIT – Learning, Research and Sharing Community.

II. Ứng dụng của vi điều khiển, sự phổ biến của dòng vi xử lý ARM Cortex M

1. Sự phổ biến

- 25% Vi điều khiển sử dụng ARM.
- Các thiết bị giám sát sức khỏe.
- IoT.
- Đo lường.
- Tìm kiếm: site: ifixit.com stm32
- Lý do:
    - Chi phí thấp, năng lượng tiêu thị thấp, diện tích nhỏ.
    - Bộ VXL 32bit cung cấp khả năng tính toán tốt.
    - VXL có hỗ trợ FPU Unit, DSP Unit, MPU, …

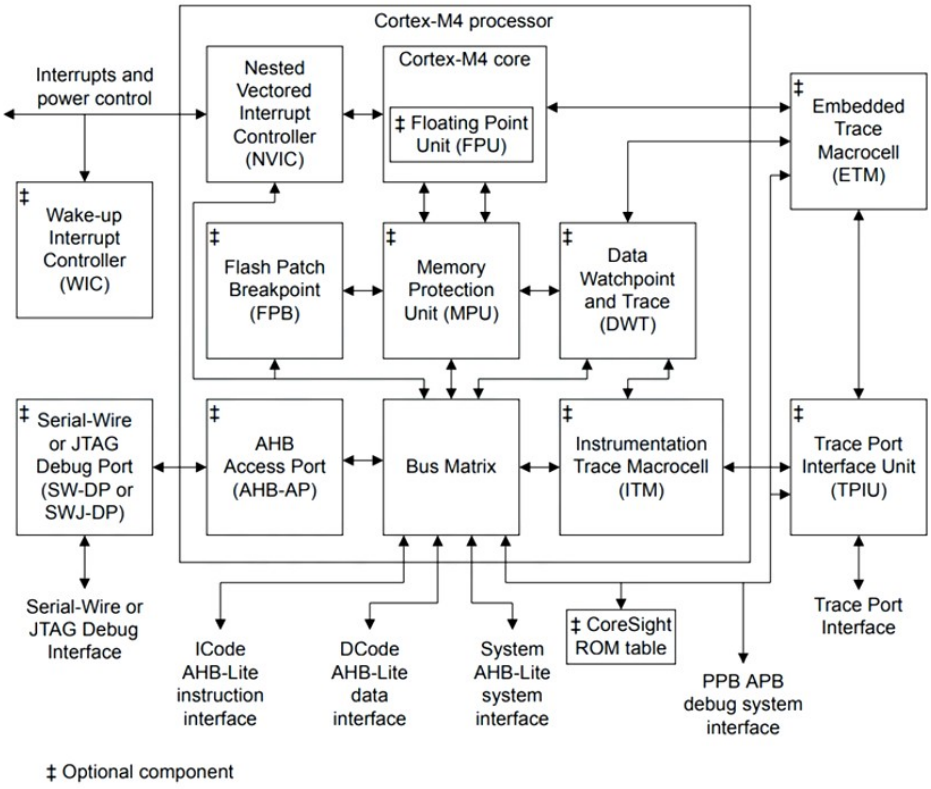
Downloaded by Du Di?p (diepdu2005@gmail.com)

- FPU: làm việc tốt hơn với số thực.
- DSP Unit: Làm việc với tín hiệu số.
- MPU: Bảo vệ bộ nhớ.

- Bộ điều khiển ngắt với khả năng hỗ trợ 240 ngắt.
- Hỗ trợ tốt cho RTOS (Hệ điều hành thời gian thực):
  - Cung cấp các inception (?)
  - Các chế độ động, truy xuất, bảo mật tốt hơn.

III. Các khái niệm CPU CORE, CPU và MCU

1. Khái niệm Processor Core và Processor



Sơ đồ cấu trúc VXL ARM Cortex M4

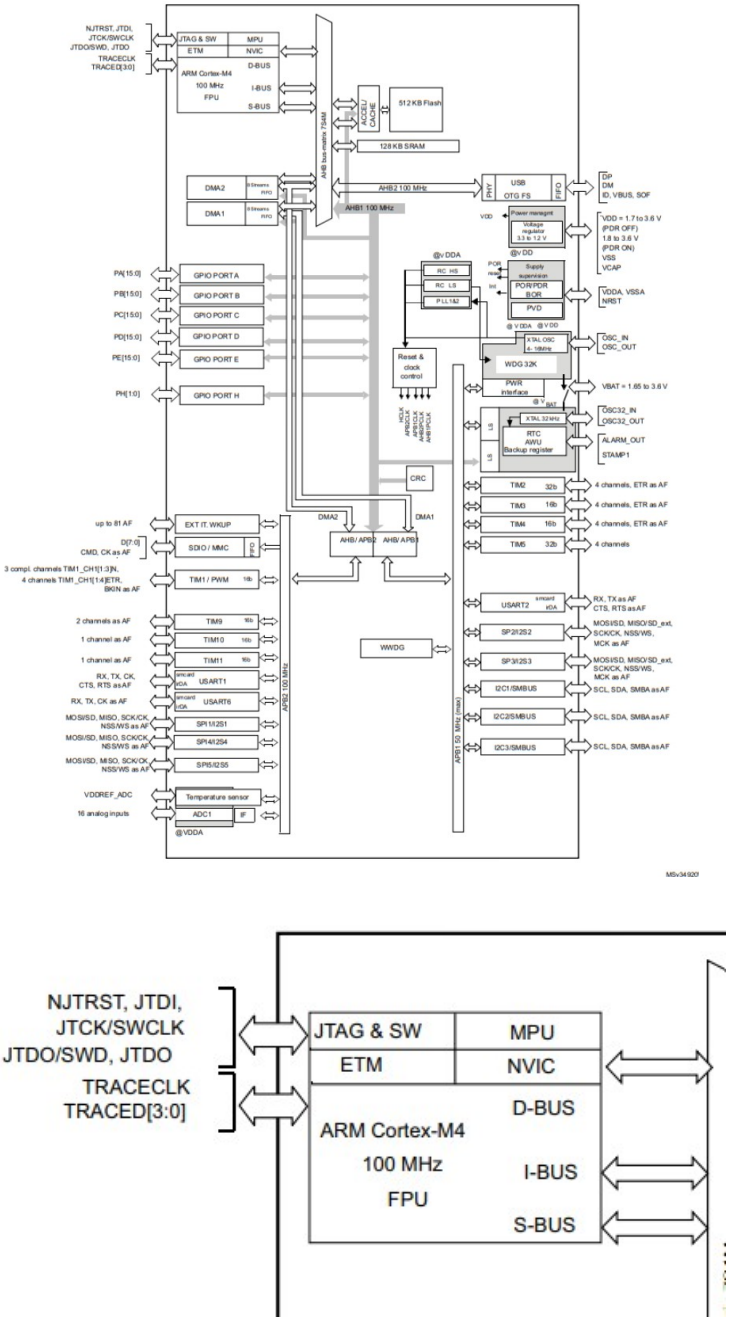
- Tài liệu: Technical Reference Manual (TRM): inforcenter.arm.com
- CPU = Processor = Vi xử lý.
- CPU Core = Processor Core = Lõi vi xử lý.
- MCU = Vi điều khiển
- Bao gồm 1 Processor con (Cortex-M4 Core) và xung quanh có nhiều ngoại vi chuyên dùng:

Downloaded by Du Di?p (diepdu2005@gmail.com)

    - NVIC (Nested Vectored Interrupt Controller):
    - FPB (Flash Patch Breakpoint):
    - MPU (Memory Protection Unit):
    - DWT (Data Watchpoint and Trace):
    - AHB-AP (AHB Access Port):
    - Bus Matrix:
    - ITM (Instrumentation Trace Macrocell):
- Cortex-M4 core bao gồm:
    - Khối tính toán số học và logic: ALU.
    - Giải mã và thực thi lệnh: CU.
    - Các thanh ghi lưu trữ dữ liệu: Register.
    - Kỹ thuật pipeline: Tăng tốc độ thực thi lệnh.
- 1 Processor có thể có 1 hoặc nhiều Core.
    - Với ARM Cortex M4: 1 Core -> Single Core Processor.
- Processor con và tất cả các ngoại vi liên quan là 1 Software IP format: 1 phần mềm và được bán. Giữ nguyên hoặc loại bỏ 1 số thành phần.

2. Khái niệm Processor và Microcontroller.

Downloaded by Du Di?p (diepdu2005@gmail.com)



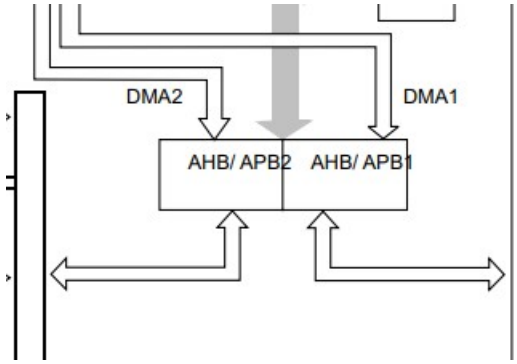
CPU

- CPU: Hoạt động như 1 Master, giao tiếp với các ngoại vi qua các Bus
  - D-Bus: thực hiện kết nối Data Bus đến ma trận Bus để tải các câu lệnh xuống hoặc truy cập debug từ bộ nhớ Flash hoặc SRAM.
  - I-Bus : thực hiện kết nối Instruction Bus đến ma trận Bus để lấy các câu lệnh từ bộ nhớ Flash hoặc SRAM.

Downloaded by Du Di?p (diepdu2005@gmail.com)

- S-Bus: kết nối System Bus đến ma trận Bus để truy cập dữ liệu trên 1 ngoại vi hoặc trên SRAM. Có thể lấy câu lệnh giống I-Bus nhưng kém hiệu quả hơn.

- Hệ thống AHB/APB: Liên kết các ngoại vi với nhau và với CPU



C. Phần cứng

I. Phân tích và lựa chọn board

1. Tiêu chí lựa chọn

- Board mạch chính hãng
- Tài liệu:
  - User Manual
  - Datasheet
  - Tích hợp sẵn Debugger (ST – Link)
    - Cấp nguồn
    - Debug
    - Nạp chương trình
  - Cộng đồng người sử dụng
  - Cấu hình ngoại vi, bộ nhớ phù hợp có được hỗ trợ hay không
    - CAN
    - Dung lượng: nên 32KB SRAM, 128KB Flash

II. Chi tiết Board Discovery và Nucleo STM32F411

- Tài liệu: User Manual

1. Nguồn

- Điện áp: Máy tính, hoặc 5V (Chân 5V)
- Dòng tiêu thụ dưới 100mA

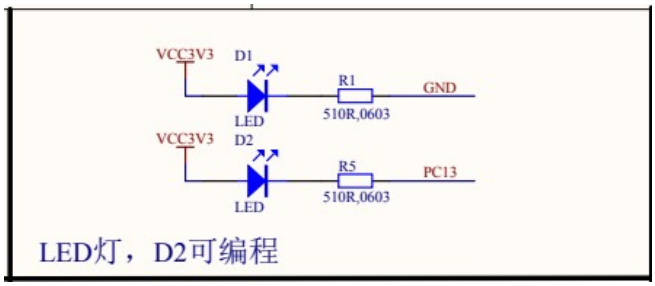
2. Led

- Mặc định đỏ
- Khi đang kết nối màu xanh

Downloaded by Du Di?p (diepdu2005@gmail.com)

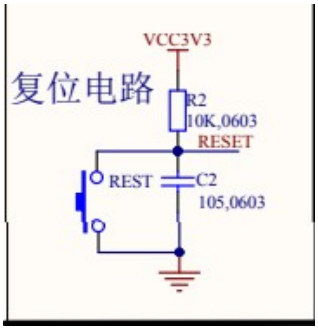
- Led PWR: Đèn led nguồn, sáng đỏ khi được cung cấp nguồn. Quan sát xem bo đã được cấp nguồn chưa
- Led USER – PC13: Kết nối với I/O ở chân C13

3. Sơ đồ nguyên lý



- Các cực Anot nối với VCC, Các cực Catot nối với chân, muốn đèn sáng thì đưa chân tương ứng về mức logic 0.

4. Button: RESET



5. Header

6. Thạch anh

- Rất gần VĐK chính: Dao động ngoại
- Mặc định dùng dao động nội

D. STM32CubeIDE

I. Tổng quan và cài đặt

1. Tính năng

- Chọn vi điều khiển hoặc board mạch để tạo project.
- Khởi tạo và cấu hình ngoại vi thông qua giao diện đồ họa.
- Tạo ra mã nguồn C từ các cấu hình.
- Phát triển Firmware.
    - Có thể xuất tài liệu pdf và toàn bộ cấu hình project.
    - Bất cứ lúc nào cũng có thể quay lại cấu hình thêm hoặc hiểu chỉnh ngoại vi, không ảnh hưởng tới mã chương trình.

Downloaded by Du Di?p (diepdu2005@gmail.com)

- Biên dịch và nạp chương trình cho vi điều khiển.
- Chức năng debug: Giá trị, vẽ đồ thị, …

2. Cài đặt

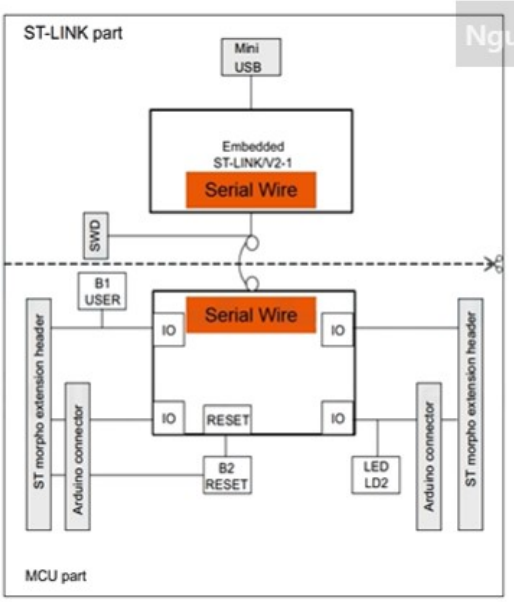
3. Tạo project

- Nền tảng Eclipse
- Sử dụng các Perspective (Giao diện)
- Device configuration tool (STM32CubeMX)
    - Khởi tạo
    - Cấu hình các ngoại vị, bộ dao động, thư viện.
    - Tạo mã chương trình C.
- C/C++
    - Viết và chỉnh sửa mã chương trình.
- Debug
    - Kiểm tra và gỡ lỗi cho chương trình.
- System Core: Ngoại vi lõi
- Multimedia: Khối xử lý âm thanh
- Computing: Tính toán kiểm tra lỗi
- Middleware: các lớp phần mềm, thư viện, giao tiếp dữ liệu, thời gian thực

*** Thiết lập

a. System Core

- Sys - Debug: Serial Wire.



Downloaded by Du Di?p (diepdu2005@gmail.com)

Kết nối để nạp chương trình lần thứ 2

b. Ngoại vi
- Lựa chọn trực tiếp

c. Clock Configuration
- Chọn các nguồn dao động thạch anh

IV. Cấu trúc của Project

1. Bố cục của chương trình C

- Header
- Include
    - Các file .h, .c
    - Typedef
    - Define
    - Macro
    - Variable
    - Function prototype: Nguyên mẫu hàm
    - Main
        - Trước while: Chỉ được thực hiện 1 lần
        - Trong While: thực hiện tuần hoàn
    - Function definition
- Viết mã chương trình giữa cặp chú thích /*User CODE BEGIN*/ và /*User CODE END*/

2. Thao tác điều khiển

- Tìm đến khai báo, định nghĩa
    - Nhấn chuột phải, ấn Open declaration
- Báo lỗi: Dấu chấm than
- Chú thích: Ctrl /
- Đi đến vị trí chỉnh sửa gần nhất: <= : Ctrl Q
- Quay về file vừa được chỉnh sửa: <=: Alt + Left
- Phóng to chữ: Ctrl + Shift + '+'
- Thu nhỏ chữ: Ctrl + '-'
- Định dạng căn lề cho mã chương trình: Ctrl + Shift + F
- Biên dịch mã chương trình

3. Driver

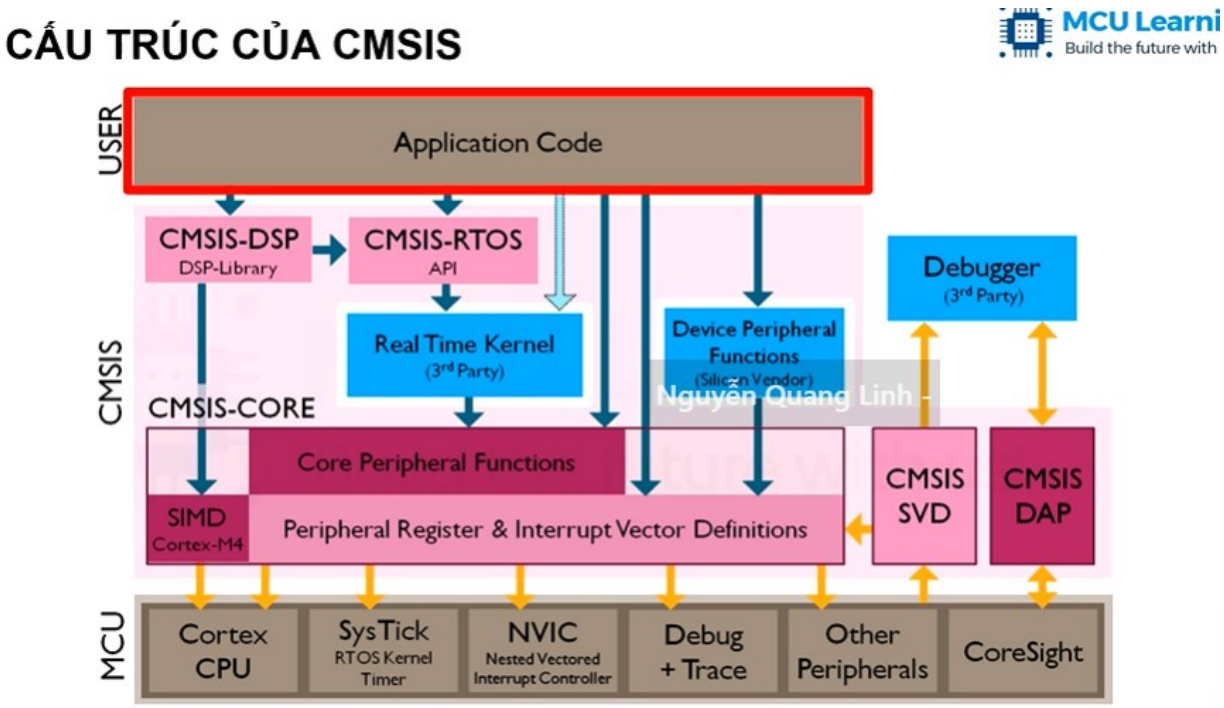
- CMSIS
- HAL: Sử dụng các ngoại vi của vi điều khiển

Downloaded by Du Di?p (diepdu2005@gmail.com)

E. Thư viện và chương trình mẫu

I. Tìm hiểu về CMSIS

- CMSIS:
  - Được cung cấp dành cho VĐK lõi ARM
  - Người dùng có thể giao tiếp



- Application: main.c
- Chạy ở VXL ở tầng MCU
- Có thể tác động qua các thanh ghi qua địa chỉ của chúng
- Application Code có thể không sử dụng lại được nữa
- Địa chỉ của thanh ghi, vai trò, vị trí bit trong thanh ghi thay đổi khi thay đổi vi điều khiển.
- CMSIS cung cấp các API cho VĐK, không cần quan tâm tới các vị trí, địa chỉ, chức năng của các thanh ghi.
- Có các thư viện khác: Device …. (STM32 Peripheral HAL)
- Code HAL -> CMSIS -> Phần cứng

II. Thư viện HAL

- HAL (Hardware Abstraction Layer):
- Trong tv Driver – HAL – Src – Các chức năng mặc định của vi xử lý.
- Cung cấp đủ các hàm để làm việc với các chế độ

Downloaded by Du Di?p (diepdu2005@gmail.com)

    - Polling
    - Interrupt
    - DMA
- Hỗ trợ timeout cho hàm blocking
- Hỗ trợ quản lý lỗi đường truyền
- Có 2 thư viện: _ex: Chứa các hàm có chức năng mở rộng cho F4
    - Ko có _ex: Dùng cho tất cả các loại VĐK

III. Chương trình mẫu

- Thư viện HAL
- GPIO
    - GPIO Input: Nhập tín hiệu số
    - GPIO Output: Xuất tín hiệu số
- Brief: Chức năng của hàm
- Param: tham số cần điền

IV. Debug

- Biên dịch trước khi vào Debug
- Thoát khỏi Debug ngay sau khi sử dụng

1. Các tính năng Debug

- Giám sát và tác động vào hoạt động của VĐK
- Phát hiện lỗi
- Tối ưu và loại bỏ lỗi
- Tìm hiểu quá trình thực hiện lệnh

2. Điều khiển quá trình thực thi của chương trình.

- Dừng ở câu lệnh nào thi câu lệnh đó chưa thực hiện.



Downloaded by Du Di?p (diepdu2005@gmail.com)

F. Kiến trúc VXL ARM

I. Chế độ hoạt động

- Thread: Thực hiện các công việc tuần tự theo chương trình chính.
- Handler: Thực hiện các exception
- Thanh ghi: xPSR
    - 8 bit cuối của IPSR
        - = 00H: Chế độ Thread
        - Khác: Chế độ Handler

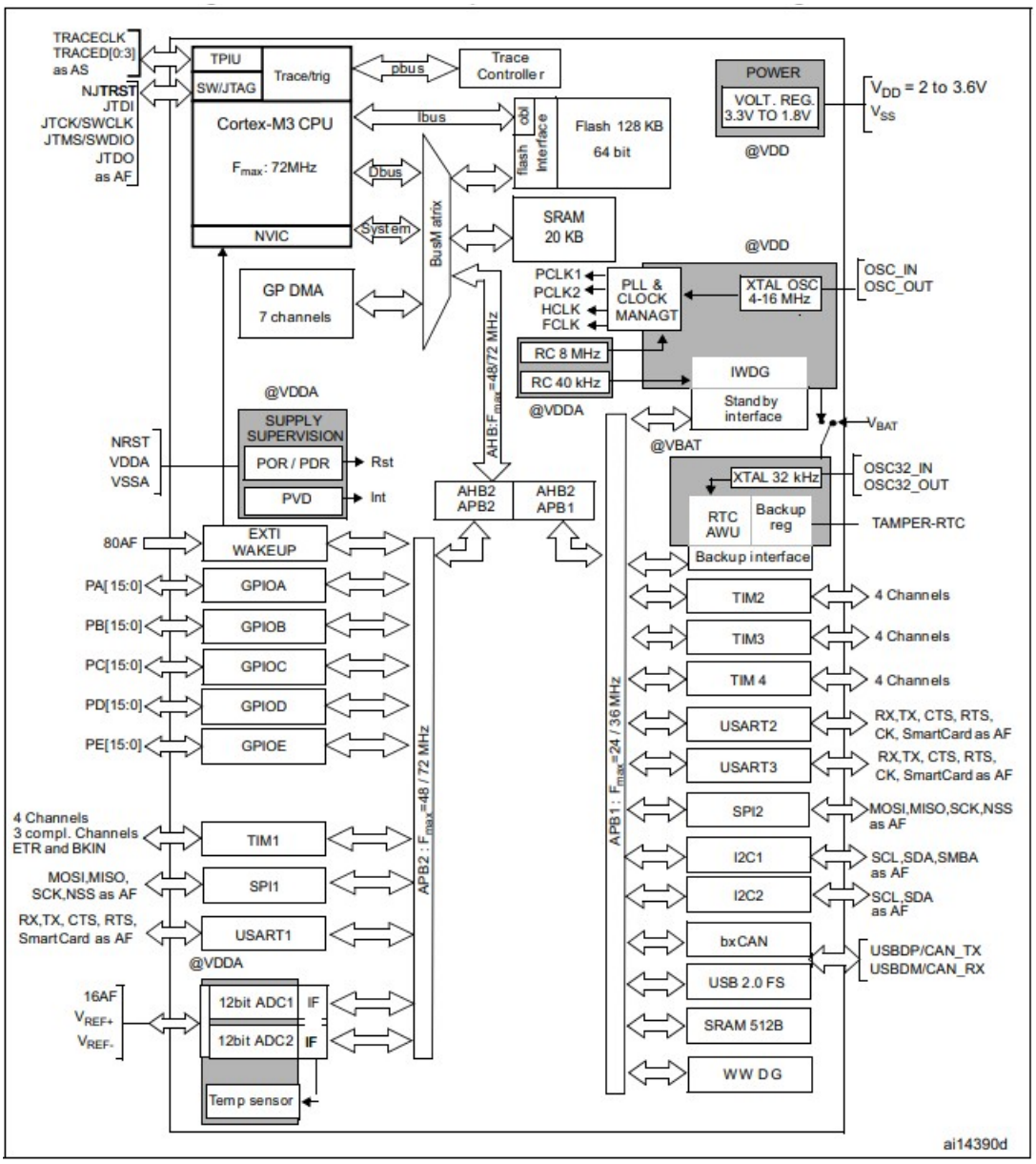
II. Các thanh ghi

- 13 thanh ghi: R0 -> R12
- Stack Pointer:
    - PSP: Process
    - MSP: Main
        - Khi nạp lại, giá trị MSP = 0x0000 0000
- LR: Lưu giá trị trả về
- PC: Lưu nội dung câu lệnh tiếp theo
- PSR: Thanh ghi trạng thái
    - Bit T: Luôn bằng 1 -> bit 0 của thanh ghi PC
    - Thực tế bit 0 của PC luôn bằng 1, nhưng trong Debug để dễ quan sát thì ko xét tới, nhưng LR thì có, nên giá trị của LR = PC +1

III. Hệ thống Bus

- AHB (Advance High Performance Bus): Bus chính
- APB (Advance Peripheral Bus):
    - Giao tiếp đơn giản
    - Năng lượng tiêu thụ thấp
    - Tốc độ chậm
- Cầu chuyển đổi AHB – APB

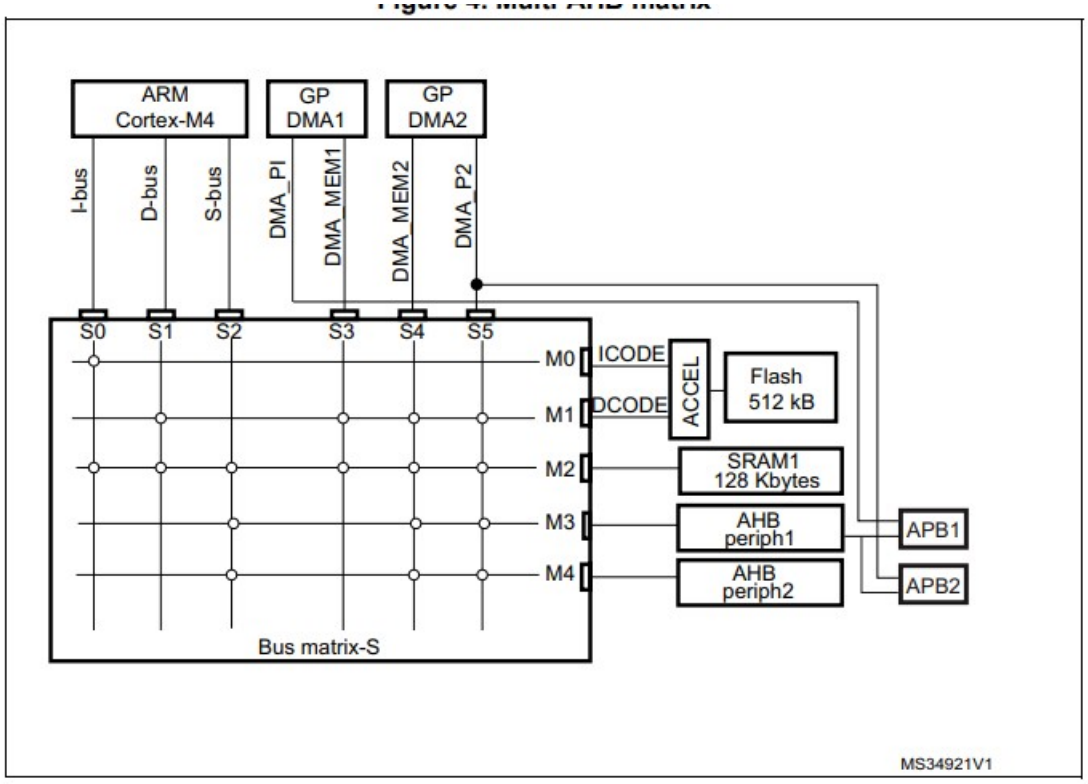
Downloaded by Du Di?p (diepdu2005@gmail.com)



- I-Bus:
  - Lấy lệnh (Fetch Instruction)
  - Điểm đến: Bộ nhớ chứa code (Flash hoặc SRAM)
- D-Bus:
  - Lấy dữ liệu hằng (literal) và truy xuất dữ liệu trong debug.
  - Điểm đến: Bộ nhớ chứa code (SRAM)
- S-Bus:
  - Truy xuất dữ liệu từ ngoại vi hoặc SRAM.
  - Điểm đến: Bộ nhớ dữ liệu SRAM và các ngoại vi kết nối vào các bus AHB, APB.

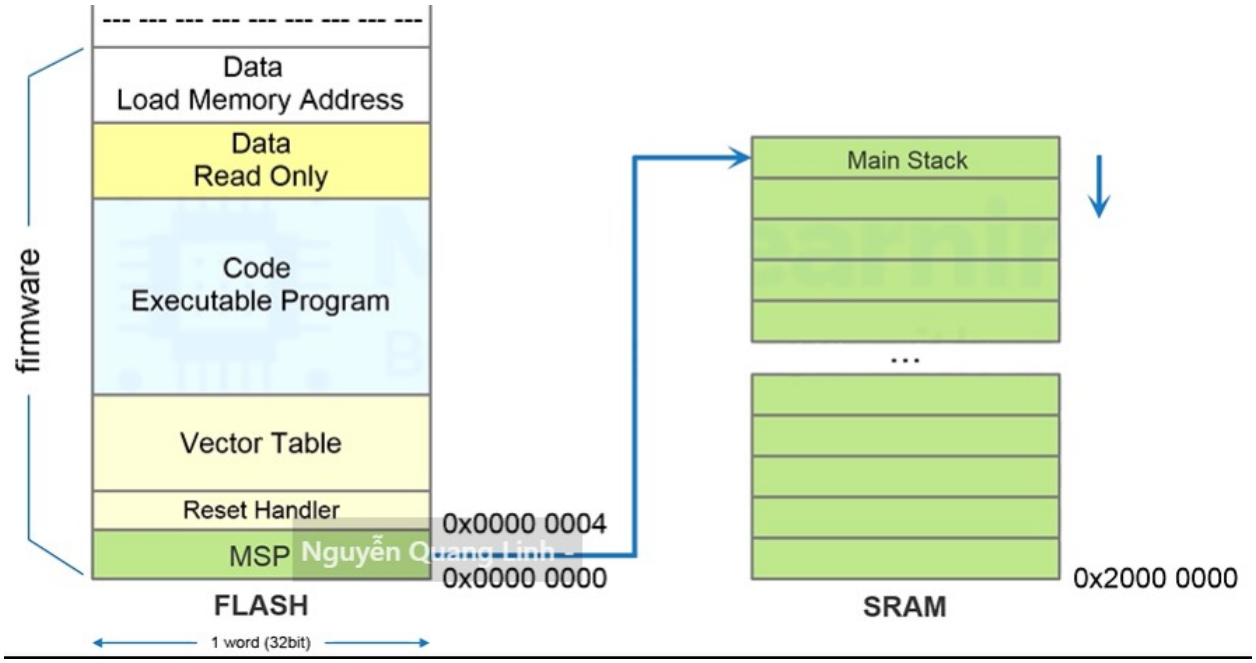
Downloaded by Du Di?p (diepdu2005@gmail.com)

- Matrix bus (M4)



IV. Trình tự khởi động

1. Chương trình trong bộ nhớ



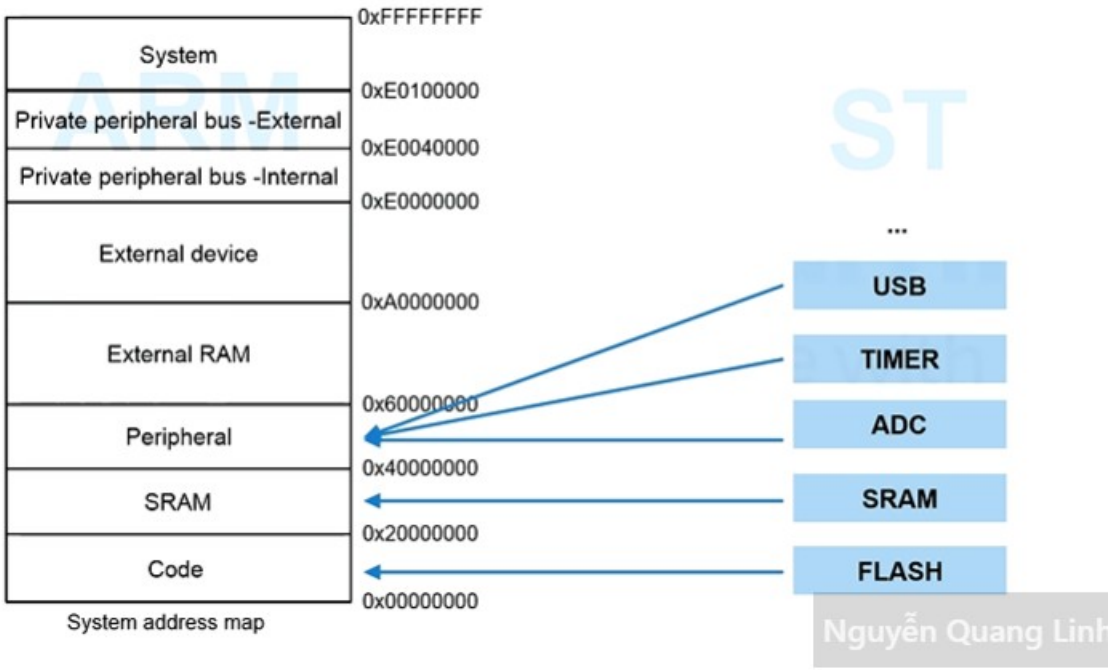
2. Quá trình khởi động của vi xử lý

Downloaded by Du Di?p (diepdu2005@gmail.com)

- Khởi tạo giá trị MSP
- Đọc giá trị tại 0000 0004 nạp cho PC
- Nhảy tới Reset Handler
    - Khởi tạo giá trị các biến (các giá trị mặc định)
    - Nhảy tới hàm Main
- Có thể RESET mềm
    - NVIC_SystemReset();
    - Địa chỉ bắt đầu bộ nhớ Flash: 0x0800 0000 (STM32 hỗ trợ kết nối với 0x0000 0000)

G. Tổ chức bộ nhớ

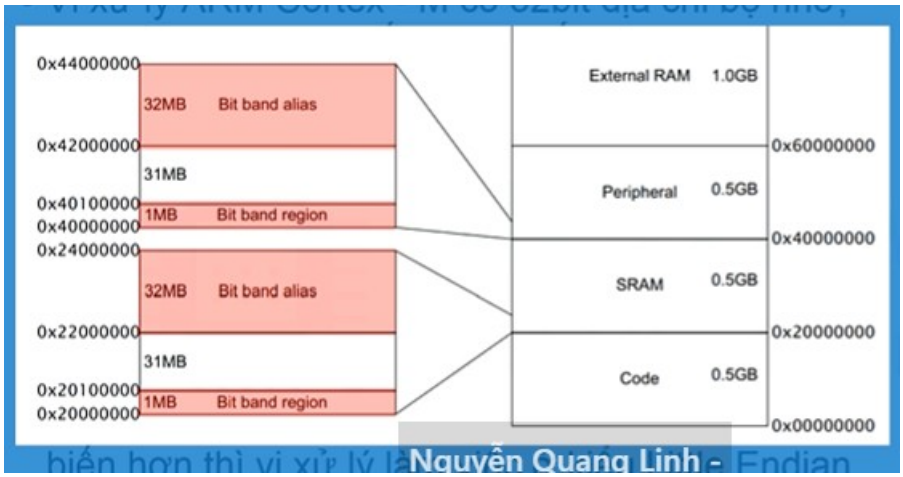
I. Chức năng bộ nhớ



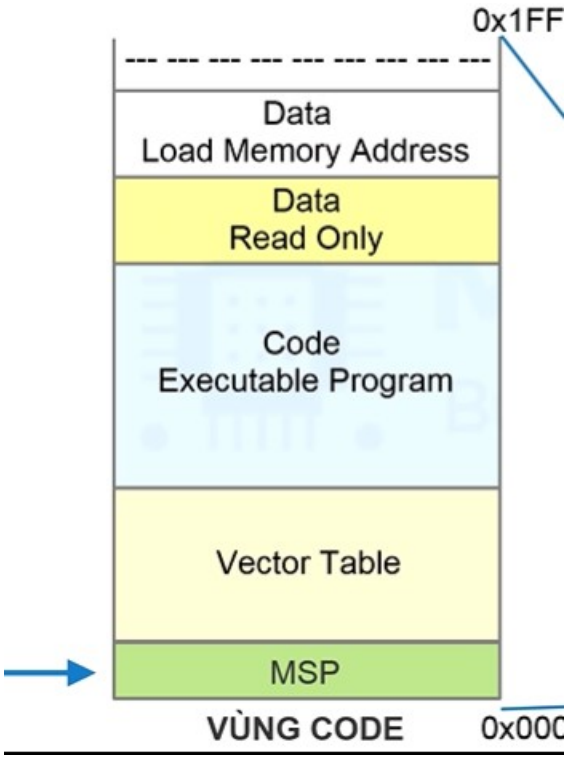
System address map

- 32 bit địa chỉ bộ nhớ: 4GB
- Đánh địa chỉ theo kiểu Little Endian và Big Endian. Mặc định là Little Endian.
- Một phần bộ nhớ có địa chỉ dành cho 1 bit dữ liệu (bit_banding) – đánh địa chỉ cho 1 bit dữ liệu riêng lẻ trong vùng nhớ này.

Downloaded by Du Di?p (diepdu2005@gmail.com)

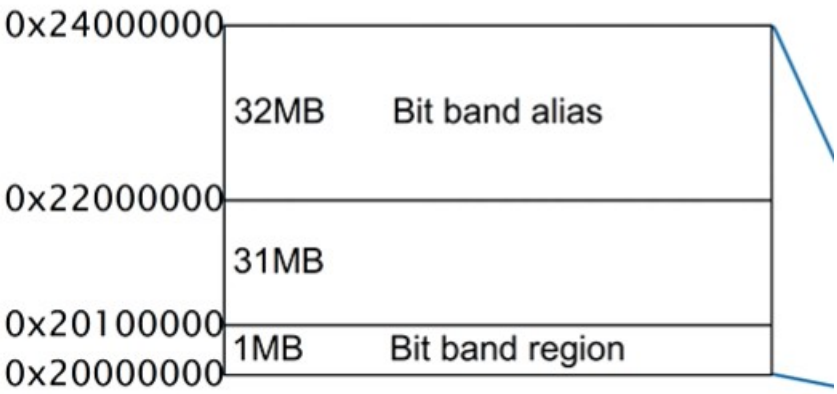


- Vùng Code: 512MB (Flash hoặc ROM)

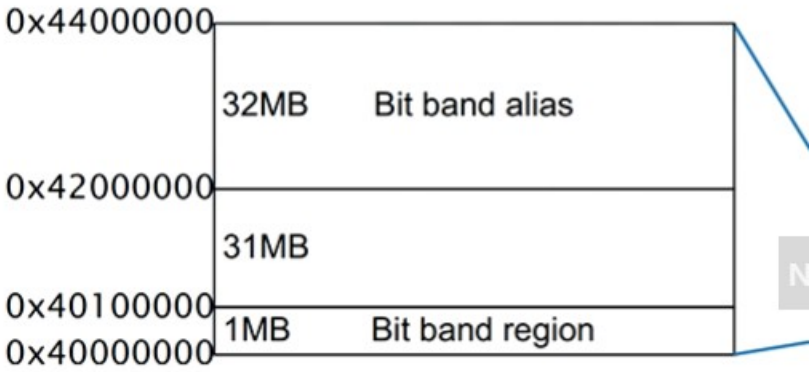


- SRAM:

Downloaded by Du Di?p (diepdu2005@gmail.com)



- Vùng Peripheral:



II. Lưu trữ và chuyển dữ liệu Alidned, Unaligned

1. Aligned Data (Căn chỉnh)

- Dữ liệu được căn chỉnh trong bộ nhớ
- Địa chỉ của dữ liệu sẽ là bội số của kích cỡ dữ liệu ở đơn bị byte
    - 1 byte: char(int8_t), unsigned char(uint8_t): không giới hạn giá trị địa chỉ.
    - 2 byte: short (int16_t), unsigned short(uint16_t): địa chỉ có giá trị cuối cùng bằng 0: 0x0020, 0x1210
    - 4 byte: int(int32_t), unsigned int(uint32_t), float: địa chỉ có giá trị của 2 bit cuối cùng bằng 00: 0x0200, 0x3200
- Ưu điểm: Đảm bảo hiệu năng làm việc (Khi cần truy xuất sẽ truy xuất trực tiếp tới byte đó)
- Nhược điểm: Lãng phí bộ nhớ

Downloaded by Du Di?p (diepdu2005@gmail.com)

C type	stdint.h type	Bits	Sign
char	uint8_t	8	Unsigned
signed char	int8_t	8	Signed
unsigned short	uint16_t	16	Unsigned
short	int16_t	16	Signed
unsigned int	uint32_t	32	Unsigned
int	int32_t	32	Signed
float		32	
pointer		32	
unsigned long long	uint64_t	64	Unsigned
long long	int64_t	64	Signed
double		64	

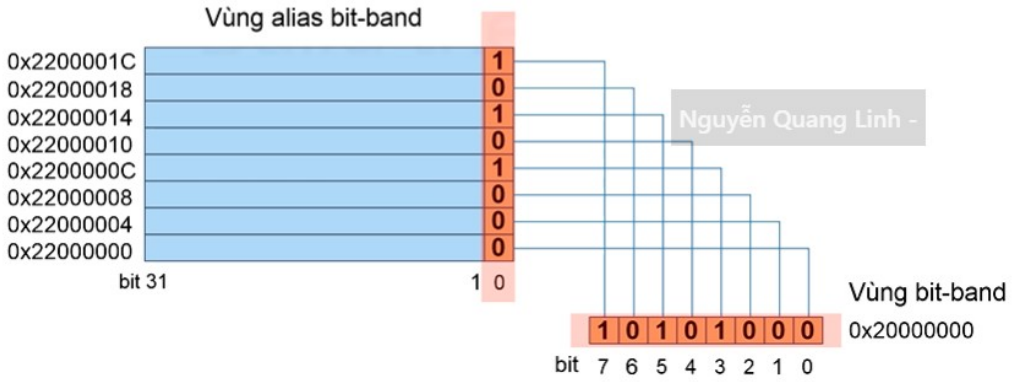
2. Unaligned Data (Không căn chỉnh): Địa chỉ lưu trữ liên tục

- Structure
- Union
- Pointer
- Sử dụng từ khóa __packed
- Ưu điểm: Không lãng phí bộ nhớ trong SRAM
- Nhược điểm: Giảm hiệu năng (Do phải truy xuất ngược lại từng byte)

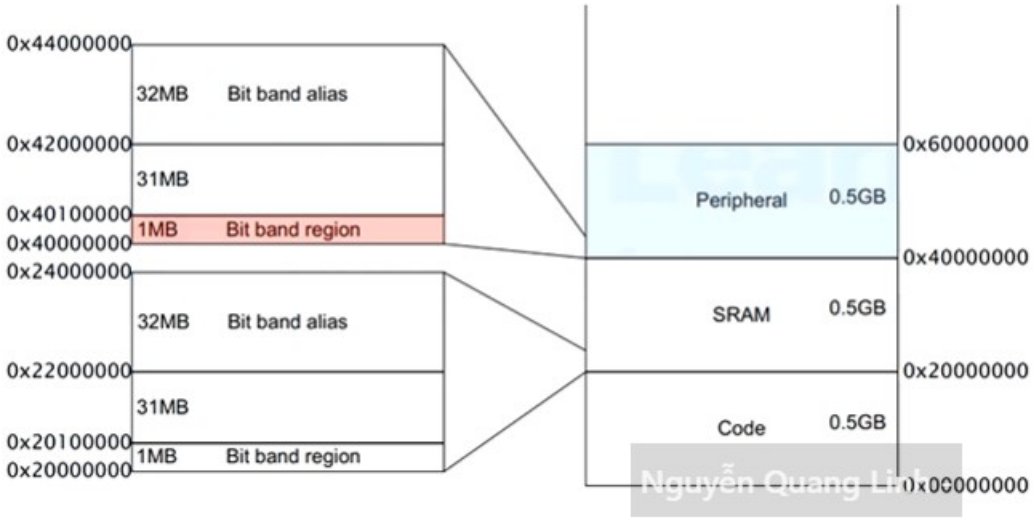
III. Bit – Banding

- Ánh xạ mỗi bit riêng lẻ trong vùng nhớ bit-band tới một word ở vùng alias bit-band trong bộ nhớ.
- Mỗi bit ở vùng nhớ bit-band sẽ có 1 word tương ứng ở vùng alias (bí danh) bit-band.
- Truy xuất vào các bit ở vùng nhớ bit-band bằng cách truy xuất vào xác word tương ứng ở vùng nhớ alias bit-band thông qua địa chỉ các word này.

Downloaded by Du Di?p (diepdu2005@gmail.com)



- Ở 2 vùng thấp nhất của SRAM và Pheripheral
- SRAM: 0x20000000 -> 0x200FFFFF …

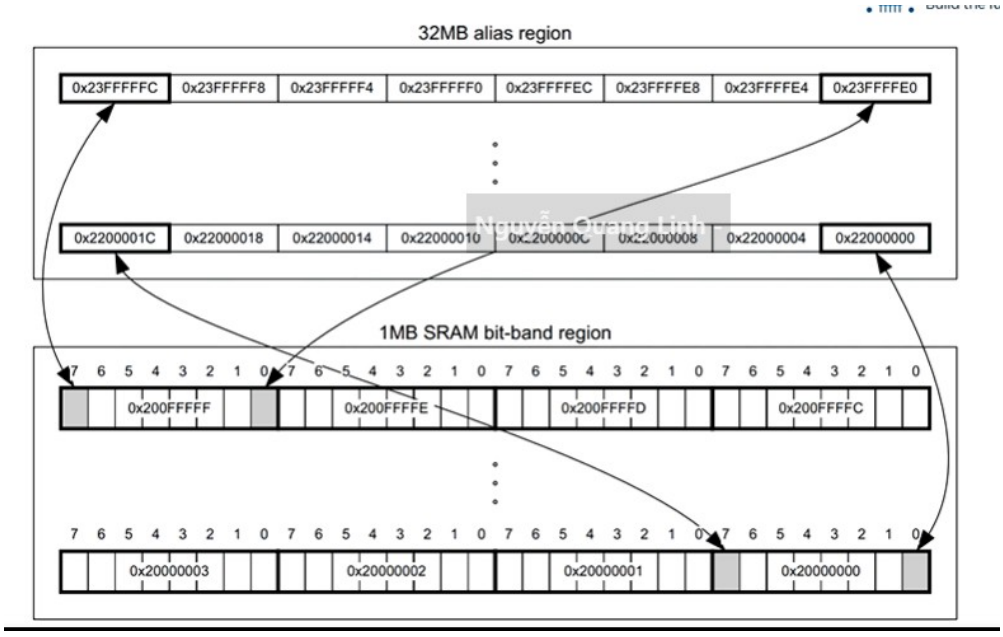


- Công thức xác định vùng alias bit-band:

Bit_word_addr = bit_band_base + (byte_offser x 32) + (bit_number x 4)

  - Bit_word_addr: địa chỉ của word ở vùng nhớ alias tương ứng với bit mà ta muốn làm việc ở vùng bit-band.
  - bit_band_base: địa chỉ bắt đầu của vùng alias bit-band.
  - byte_offset: số thứ tự của byte chứa bit cần làm việc trong vùng bitband.
  - bit_number: vị trí của bit muốn làm việc (0 – 7).

Downloaded by Du Di?p (diepdu2005@gmail.com)



H. System Exceptions and Interrupt

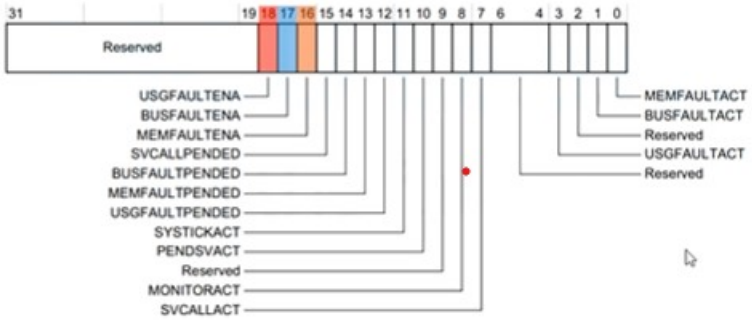
I. System Exception

- Exception là một điều kiện làm thay đổi thứ tự thực hiện bình thường của một chương trình.
    - Thay đổi chế độ hoạt động của vi xử lý từ chế độ Thread sang Handler.
- 2 loại:
    - System Exception: Tạo bởi VXL
    - Interrupt: Ngoại vi
- Reset: Khi cấp nguồn cho VĐK, bằng tay, phần mềm
    - Reser Handler được thực thi.
    - Có độ ưu tiên cao nhất.
- Fault Exception: Phát hiện lỗi
    - HardFault: Luôn được bật, cho phép thực thi Handler tương ứng là HardFault Handler.
        - Xảy ra:
            - Có lỗi trong quá trình thực thi các Fault Handler khác như: MemManage Handler, BusFault Handler và UsageFault Handler.
            - Có lỗi mà Fault Exception Handle tương ứng bị tắt đi thì HardFault được kích hoạt thay thế.

Downloaded by Du Di?p (diepdu2005@gmail.com)

- MemManageFault Exception: xảy ra khi có lỗi liên quan tới bảo vệ bộ nhớ.
  - Khối MPU (Memory Protection Unit) được bật.
  - VD: Exception này kích hoạt khi VXL cố truy cập để thực hiện lệnh ghi vào một vùng nhớ được cấu hình chỉ cho phép đọc.
- BusFault Exception: Xảy ra vì các lỗi liên quan tới việc truy cập vào vùng nhớ dữ liệu hoặc vùng nhớ chương trình.
  - VD: Truy cập dữ liệu tại một địa chỉ sai. Hoặc khi tràn Stack.
- UsageFault: Xảy ra khi liên quan tới thực thi các câu lệnh.
  - VD: Chia cho 0, có con trỏ hàm trỏ tới một địa chỉ bị sai.
- SysTick Exception: Tạo ra bởi System Timer khi đạt giá trị 0.
  - Systick Handler được sử dụng để tạo khoảng thời gian trễ (delay)
  - Trong RTOS, thường dùng để chuyển đổi ngữ cảnh.

- Lưu ý: Mặc định HardFault Handler được bật, còn lại thì tắt. Khi các Fault còn lại xảy ra thì đều dẫn đến HardFault Exception.
- Tập SCB (System Control Block)
  - SHCSR: Cho phép các Fault Exception Handler



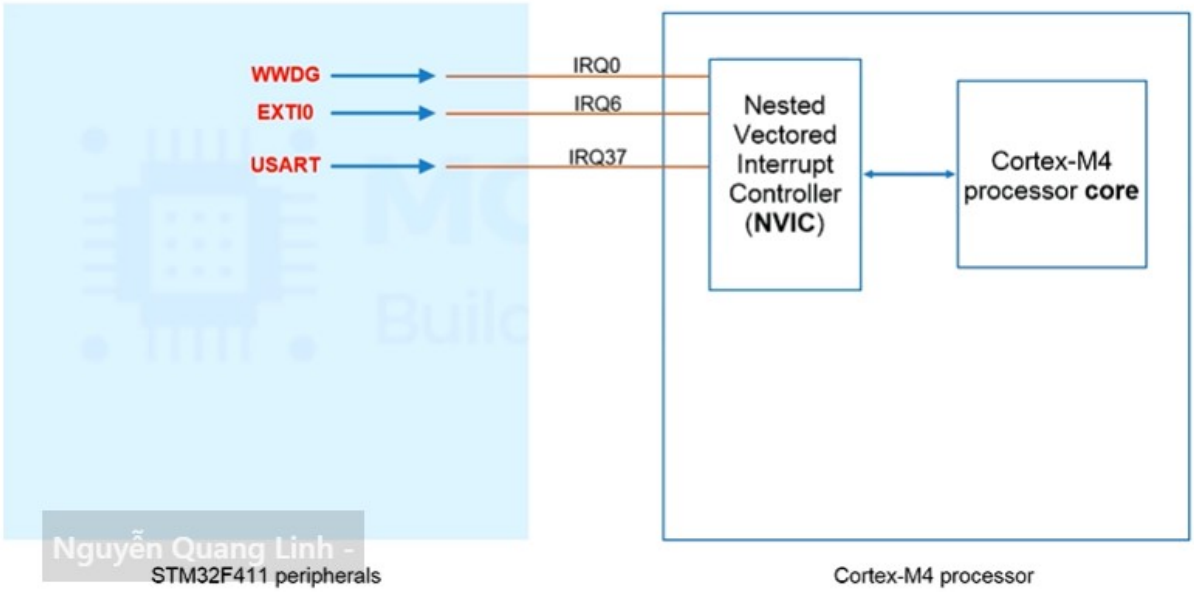
  - CFSR: Chứa các lỗi thực thi
- Câu lệnh Setbit:
  - SCB->SHCSR |= 0b010000000000000;

II. Interrupt

- Các tín hiệu Interrupt được tạo ra bởi các ngoại vi gửi tới VXL: Interrupt Request (IRQ).
- Gửi tới NVIC (Nested Interrupt Vector Controller) của VXL.
- Vector Table: Table 63.
- Cột 1: 0 -> 59: IRQ

1. Tổng quan về thiết kế ngắt

Downloaded by Du Di?p (diepdu2005@gmail.com)



2. Vấn đề

- NVIC: Hiểu cấu hình và cấu hình độ ưu tiên.
- Ngoại vi: Sử dụng ngắt phù hợp, ngoại vi sinh ra ngắt phù hợp
- Chương trình

II. NVIC

- Cấu hình
  - Enable
  - Disable
  - Pend: Thay đổi trạng thái chờ
- Đọc trạng thái
- Độ ưu tiên

1. Các thanh ghi

- ISER: Set enable

Downloaded by Du Di?p (diepdu2005@gmail.com)

Address	Name	Type	Reset	Description
0xE000E004	ICTR	RO	-	*Interrupt Controller Type Register, ICTR*
0xE000E100 - 0xE000E11C	NVIC_ISER0 - NVIC_ISER7	RW	0x00000000	Interrupt Set-Enable Registers
0xE000E180 - 0E000xE19C	NVIC_ICER0 - NVIC_ICER7	RW	0x00000000	Interrupt Clear-Enable Registers
0xE000E200 - 0xE000E21C	NVIC_ISPR0 - NVIC_ISPR7	RW	0x00000000	Interrupt Set-Pending Registers
0xE000E280 - 0xE000E29C	NVIC_ICPR0 - NVIC_ICPR7	RW	0x00000000	Interrupt Clear-Pending Registers
0xE000E300 - 0xE000E31C	NVIC_IABR0 - NVIC_IABR7	RO	0x00000000	Interrupt Active Bit Register
0xE000E400 - 0xE000E41F	NVIC_IPR0 - NVIC_IPR59	RW	0x00000000	Interrupt Priority Register

    - Cấu hình enable khi ghi
    - Kiểm tra trạng thái Enable khi đọc
- ICER: Clear Enable
    - Ghi: 1: Disable; 0: Enable
- ISPR và ICPR
    - Ghi: Đưa một interrupt vào trạng thái Pending hoặc xóa khỏi trạng thái Pending.
    - Đọc: Biết được trạng thái Pending các Interrupt
    - Khi có nhiều Interrupt đang chờ để được xử lý thì VXL sẽ kiểm tra độ ưu tiên. Sau đó thực hiện ISR có độ ưu tiên cao nhất.
- IABR: Báo Ngắt đó đang được xử lý
- IPR: Mức độ ưu tiên

III. Interrupt Priority

- Thanh ghi: IPR: 4 bit cao: 15 mức ưu tiên
- Số càng nhỏ độ ưu tiên càng cao.
    - Preemption priority
    - Sub-priority
- Có thể ko có Preemption priority hoặc Sub-priority.
- Preemption priority: Mức độ ưu tiên preemption priority quyết định khả năng giành quyền được thực thi của một trình phục vụ ngắt.
    - Ngắt nào có Preemtion Priority cao hơn sẽ được ưu tiên thực hiện trước.

Downloaded by Du Di?p (diepdu2005@gmail.com)

- Sub-priority: Mức độ ưu tiên sub priority xác định ISR nào sẽ được thực thi trước trong trường hợp có nhiều ISR cùng giá trị preemption priority đang chờ để được thực thi (Pending).

IV. Pending Interrupt

- Một ngắt đang ở trạng thái chờ cho đến khi VXL thực hiện.
- Một ngắt có thể vừa Pending vừa Active, khi đang Pending lại có tín hiệu yêu cầu ngắt đó.
- Thanh ghi ISPR và ICPR

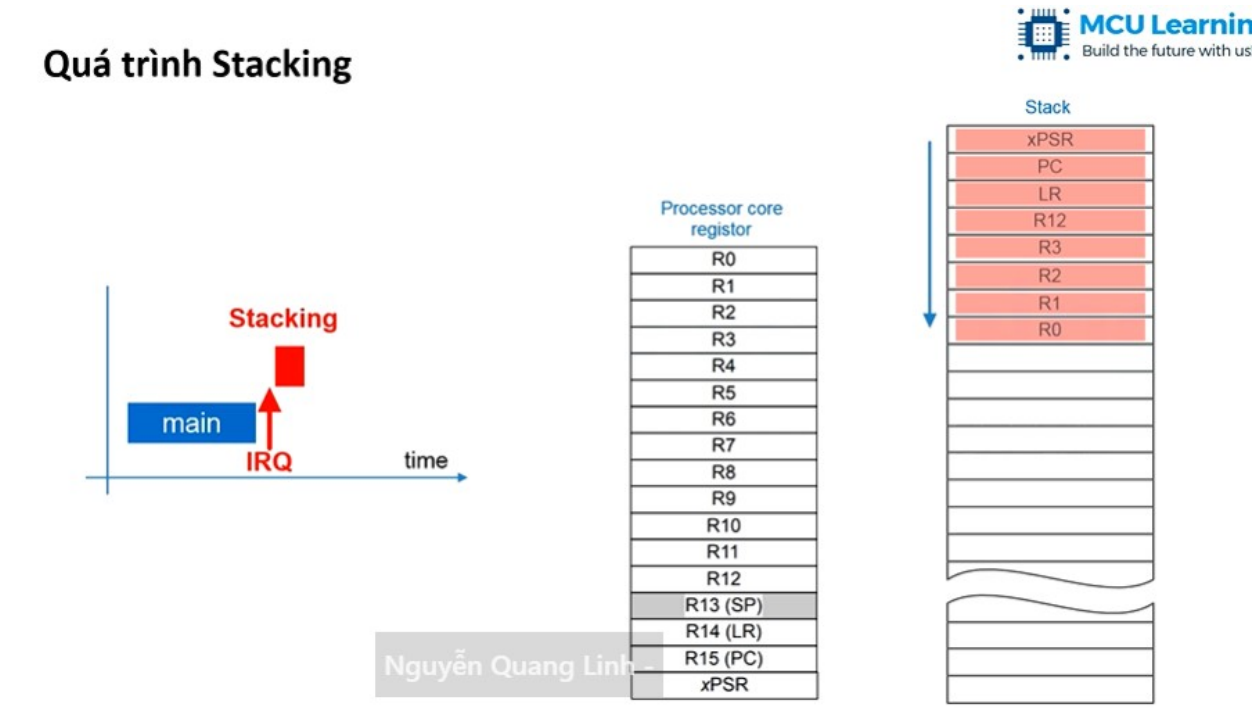
V. Quy trình vào ra ngắt của VXL

- Thư viện HAL cung cấp hàm Callback cho các ngắt.
- Địa chỉ các vector ngắt: Vector Table: Lưu trữ danh sách địa chỉ của ISR, mỗi phần tử gồm 32 bit.
- Địa chỉ lưu trữ địa chỉ của ISR tương ứng với IN n:

$$Add = 64 + 4*n$$

VI. Stacking và Unstacking khi xử lý ngắt

- Stacking: Quá trình lưu trữ ngữ cảnh, trạng thái hoạt động hiện tại của vi xử lý trước khi nhảy đến và thực thi một Interrupt Handler.
- Unstacking: Quá trình trả lại ngữ cảnh sau khi thực hiện xong một Interrupt Handler để đưa vi xử lý trở về và tiếp tục thực hiện công việc trước đó.

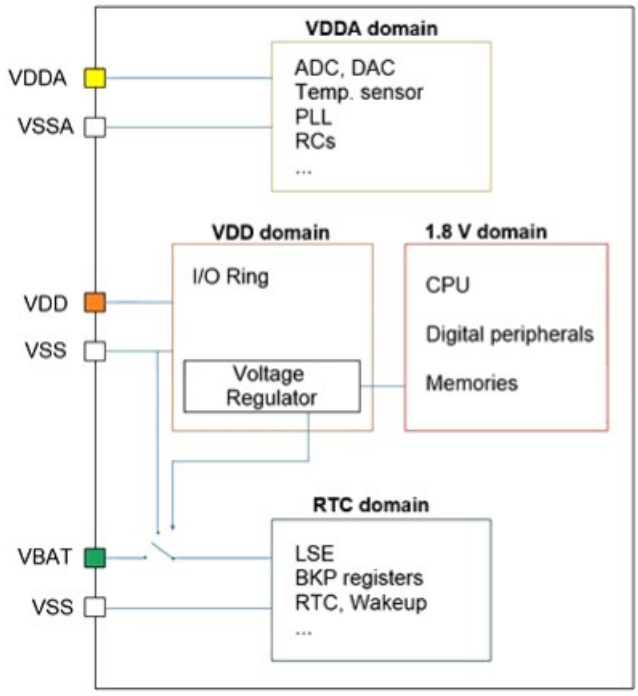


Downloaded by Du Di?p (diepdu2005@gmail.com)

- Địa chỉ của SP giảm đi 4 khi lưu đủ 8 bytes
- LIFO

J. Nguồn cấp và bộ dao động

I. Khối nguồn



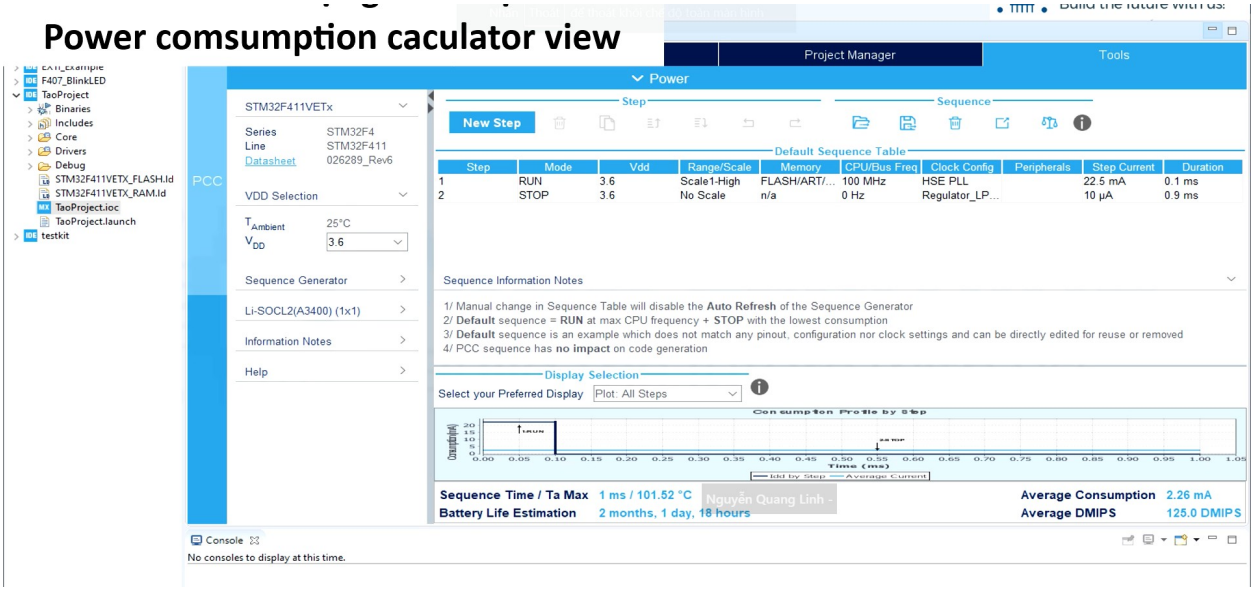
- VDDA/VSSA: Nguồn nâng cao hoạt động cho khối Analog như ADC.
    - Chân cung cấp độc lập để người dùng có thể thiết kế bộ lọc, chống nhiễu riêng.
- VDD/VSS: Cung cấp nguồn cho VDD domain từ 1.8 – 3.6V
    - Thường sử dụng 3V và 3.3V.
    - VDD cung cấp điện cho các chân I/O.
    - Khối Voltage Regulator chuyển điện áp VDD thành 1.8V để cung cấp cho 1.8V domain.
    - Cấu hình được ở chế độ tiết kiệm năng lượng.
- VBAT/VSS: Duy trì giá trị của các thanh ghi backup và cung cấp nguồn cho khối chức năng RTC.
    - Khi có nguồn cấp VDD thì RTC domain này hoạt động nhờ nguồn VDD, khi mất VDD thì chuyển sang sử dụng VBAT.
- Programmable voltage detector (PVD): Theo dõi điện áp VDD, so sánh với PVD.

Downloaded by Du Di?p (diepdu2005@gmail.com)

- VDD>PVD hoặc VDD<PVD thì gây ra ngắt EXTI line16.
- Sử dụng Interrupt Handler để đưa ra các thông báo về điện áp hoặc tắt an toàn thiết bị.

- Lưu ý:
  - Mặc dù dải điện áp từ 1.8V -> 3.6V nhưng tùy vào điện áp của nguồn cung cấp mà một vài tính năng có thể bị hạn chế nếu điện áp nguồn cấp thấp.
  - Tài liệu: "General operating conditions" trong datasheet.
  - CubeMX: Power consumption caculator view

Power comsumption caculator view



II. Chế độ tiết kiệm năng lượng

- Linh kiện
- Thiết kế nguồn
- Firmware đảm bảo tối thiểu hóa năng lượng tiêu thị của toàn bộ thiết bị.

1. Các tác nhân ảnh hưởng năng lượng

- Tốc độ Clock: Tần số hoạt động càng cao, VXL chạy càng nhanh thì càng tốn năng lượng.
  - Có mạng lưới phân phối Clock
- Sự phức tạp: Càng hỗ trợ nhiều ngoại vi càng tốn năng lượng.
  - Tắt ngoại vi khi không sử dụng.
  - STM32 cho phép tắt bất cứ ngoại vi qua hàm HAL.
- Thiết kế VĐK: Có ngoại vi hỗ trợ tiết kiệm năng lượng.
  - Cân nhắc sử dụng, lựa chọn VĐK phù hợp.

Downloaded by Du Di?p (diepdu2005@gmail.com)

- Sử dụng các chế độ tiết kiệm năng lượng, chỉ thức dậy để làm các công việc cần thiết.

2. Các chế độ hoạt động (M4)

- Run
- Sleep
- Stop
- Standby
- Những hạn chế về tính năng (Effect)
- Cách đưa VĐK vào chế độ (Entry)
- Cách thoát ra và thời gian đánh thức (Wakeup)

*** SLEEP

Effect	Entry	Wakeup
- CPU sẽ bị dừng lại. - Tất cả các I/O giữ nguyên trạng thái như lúc ở chế độ RUN. - Tất cả các ngoại vi đều thức và có thể sinh yêu cầu ngắt. - Toàn bộ dữ liệu của các thanh ghi và RAM không bị ảnh hưởng.	- Sử dụng câu lệnh ASMB WFI hoặc WFE. - Thư viện HAL_pwr: HAL_PWR_EnterSLEEPMode(,)	- Nguồn đánh thức: Any Interrupt. - VXL tiếp tục thực hiện chương trình tại vị trí Sleep. - Thời gian chuyển từ Sleep sang RUN là ngay lập tức.

- Sleep on exit: Chương trình chỉ hoạt động bên trong Interrupt Handler.
    - VDK vào chế độ Sleep, thức dậy khi có một Interrup, thực thi xong ngắt thì vào lại Sleep.
    - HAL_PWR_EnableSleepOnExit(): bật
    - HAL_PWR_DisableSleepOnExit(): tắt

- *** STOP

Effect	Entry	Wakeup

Downloaded by Du Di?p (diepdu2005@gmail.com)

- CPU sẽ bị dừng lại. - Tất cả các I/O giữ nguyên trạng thái như lúc ở chế độ RUN. - Tất cả các clock trong miền 1.8V đều bị ngừng. - Bộ giao động HIS, HSE, PLL bị Disable. - Toàn bộ dữ liệu của các thanh ghi và RAM không bị ảnh hưởng. - Khối Voltage Regulator có thể cấu hình ở chế độ Normal hoặc Low Power.	- Thư viện HAL_pwr: HAL_PWR_EnterSTOPMode(,)	- Nguồn đánh thức: Any EXTI line. - VXL tiếp tục thực hiện chương trình tại vị trí Stop. - Nguồn Clock của VĐK đưa về mặc định là HSI. - Thời gian chuyển từ STOP sang RUN phụ thuộc vào thời gian khởi động HIS, Flash, Voltage Regulator.

*** Standby

Effect	Entry	Wakeup
- CPU sẽ bị dừng lại. - Tất cả các clock trong miền 1.8V đều bị ngừng. - Bộ giao động HIS, HSE, PLL bị Disable. - Toàn bộ dữ liệu của các thanh ghi và RAM bị mất, ngoại trừ một số thanh ghi ở vùng Backup Register. - Khối Voltage Regulator có thể cấu hình ở chế độ Normal hoặc Low Power.	- Thư viện HAL_pwr: HAL_PWR_ EnterSTANDBYMode(,)	- Nút Reset. - IWDG reset. - Sườn lên nếu sử dụng WKUPx Pin. - Sự kiện RTC Alarm. - VXL chạy lại từ đầu. - Nguồn Clock của VĐK đưa về mặc định là HSI.

Downloaded by Du Di?p (diepdu2005@gmail.com)

III. Clock tree

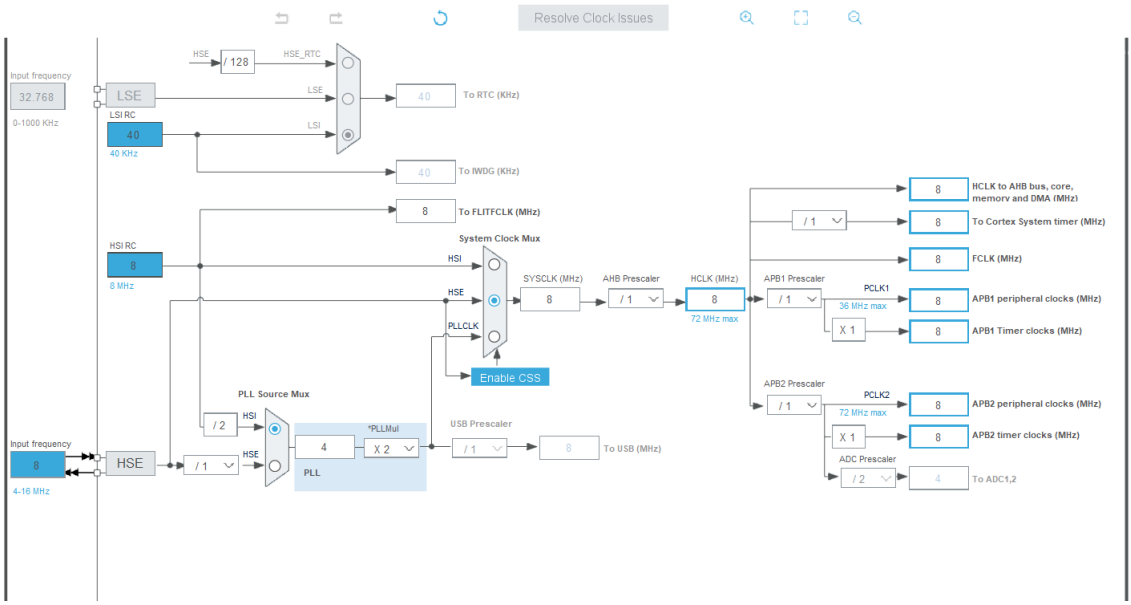
- Tín hiệu Clock được sử dụng để đồng bộ hoạt động của các mạch điện bên trong các IC số hoặc để đồng bộ IC với các mạch khác ở bên ngoài.
- Tín hiệu Clock là một tín hiệu dao động định kì.
- Tần số, đơn vị Hz.
- Liên quan tới hiệu năng và tiêu thụ năng lượng.

1. Nguồn cấp Clock

- High Speed: Nguồn clock chính.
    - Bộ dao động nội RC, gọi là High Speed Internal – HSI.
    - Bộ dao động thạch anh bên ngoài, gọi là High Speed External – HSE.
    - Lựa chọn thạch anh ngoại hay bộ dao động nội RC:
        - Độ chính xác của thạch anh cao hơn so với RC
            - RC: độ chính xác 1%.
            - Thạch anh: độ chính xác phần triệu ppm.
        - Một số ngoại vi tốc độ cao chỉ có thể chạy được với thạch anh ngoài với một tần số nhất định.
- Low Speed: sử dụng RTC (Khối thời gian thực) và IWDG.
    - Bộ dao động nội RC tốc độ thấp chuyên dụng, gọi là Low Speed Internal – LSI.
    - Bộ dao động thạch anh ngoài tốc độ thấp, Low Speed External – LSE.

2. Clock tree

- Tần số các nguồn dao động không truyền trực tiếp tới VXL hay ngoại vi.



Downloaded by Du Di?p (diepdu2005@gmail.com)

- Một số thành phần:
    - Nguồn cấp Clock: HIS, HSE, LSI, LSE.
    - Mạch chọn kênh: Lựa chọn đường tín hiệu Clock.
    - Bộ chia tần Prescaler: Giảm tần số
    - Vòng lặp khóa pha PLL: Tăng tần số

3. Cấu hình Clock Tree.

- Reset and Clock Control (RCC)
    - Cấu hình nguồn HSE và LSE.
        - Disable: Không sử dụng
        - Crystal/ Ceramic Resonator: Một bộ cộng hưởng thạch anh hoặc gốm được sử dụng thông qua các chân RCC_OSC_IN và RCC_OSC_OUT.
        - BYPASS Clock Source: Một nguồn Clock bên ngoài được sử dụng: Chân MCO của ST_Link như một HSE.

K. Thiết kế luồng xử lý của chương trình.

I. Polling

- Liên tục kiểm tra trạng thái của một hoạt động nào đó như kết quả của một điều kiện: giá trị một biến, giá trị một bit, một cờ flag nào đó.
- Liên tục thực thi một khối lệnh nào đó: Điều khiển chân Output, truyền dữ liệu qua các chuẩn giao tiếp.
- Dù cho sự kiện mong muốn có xảy ra không thì VĐK cũng liên tục kiểm tra.
- Phù hợp với ứng dụng đơn giản, không nhiều ngoại vi.
- Vòng lặp vô hạn, tuần tự.
- Ưu điểm:
    - Đơn giản, dễ Dubug
- Nhược điểm:
    - Có thể bị lỡ các sự kiện.
    - Hiệu năng thấp.

Downloaded by Du Di?p (diepdu2005@gmail.com)

II. Interrupt

- Cấu hình cho phép ngắt và độ ưu tiên ngắt ở khối NVIC qua giao diện CubeMX
- Khai báo sử dụng ngắt hoặc ngưng sử dụng ngắt ở ngoại vi thông qua các hàm của thư viện HAL

EXTI (stm32f4xx_hal_exti.c):

```
HAL_StatusTypeDef HAL_EXTI_ClearConfigLine(EXTI_HandleTypeDef *hexti)
HAL_StatusTypeDef HAL_EXTI_SetConfigLine(EXTI_HandleTypeDef *hexti, EXTI_ConfigTypeDef *pExtiConfig)
```

UART Receive (stm32f4xx_hal_uart.c):

```
HAL_StatusTypeDef HAL_UART_Receive_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)
HAL_StatusTypeDef HAL_UART_AbortReceive_IT(UART_HandleTypeDef *huart)
```

ADC (stm32f4xx_hal_adc.c):

```
HAL_StatusTypeDef HAL_ADC_Start_IT(ADC_HandleTypeDef* hadc)
HAL_StatusTypeDef HAL_ADC_Stop_IT(ADC_HandleTypeDef* hadc)
```

- Interrupts Service Routine được viết bên ngoài hàm main thông qua các hàm Callback của thư viện HAL

EXTI (stm32f4xx_hal_gpio.c):

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
```

UART (stm32f4xx_hal_uart.c):

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
```

ADC Receive (stm32f4xx_hal_adc.c):

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
```

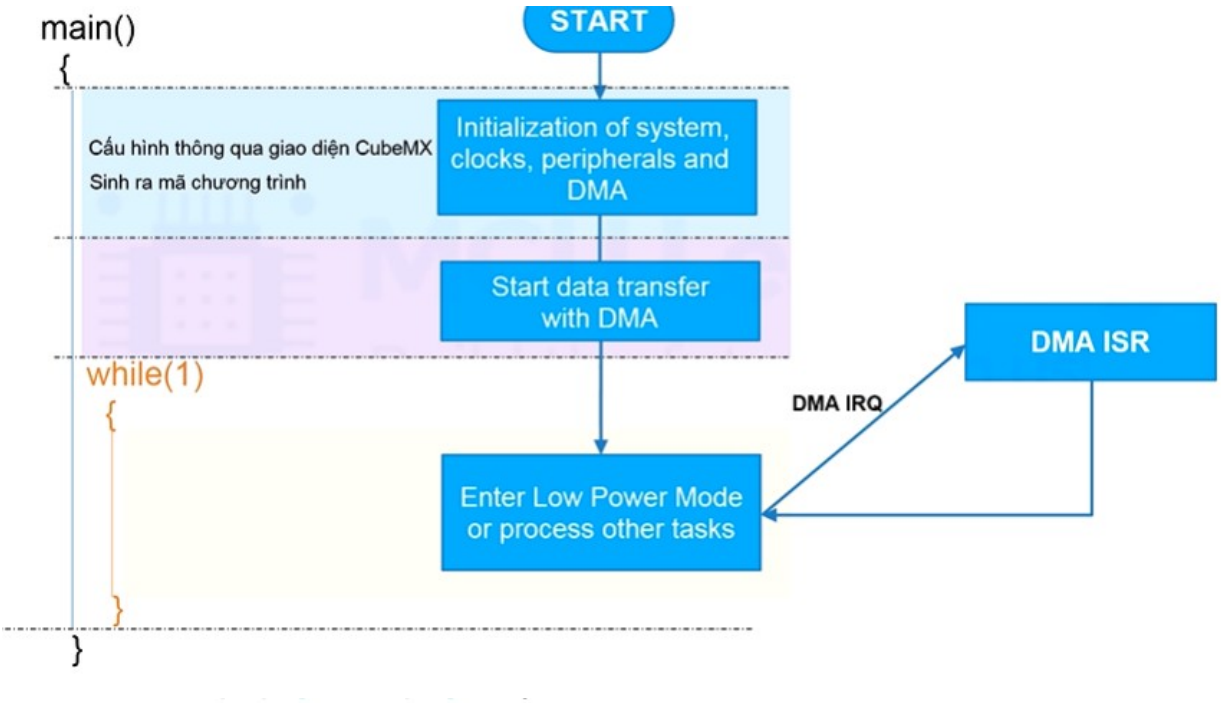
- Ưu điểm:
  - Đáp ứng nhanh
  - Tiết kiệm năng lượng.
- Nhược điểm:
  - Cần được thiết kế ngắn để tránh CPU phục vụ một ngắt quá lâu.
  - Debug phức tạp.

III. DMA

- Master kết nối với các Slave: Flash, SRAM, ngoại vi
- Truyền dữ liệu từ ngoại vi tới bộ nhớ, ngược lại mà không cần VXL.
- Các thông số:
  - Địa chỉ nguồn, đích.
  - Kiểu dữ liệu (data with): byte, half-wordm word.
  - Lượng dữ liệu (transfer size)
  - Kiểu truyền dữ liệu (transfer types)
    - Normal mode: Truyền từ nguồn tới đích, khi đủ lượng dữ liệu thì dừng lại.
    - Circular mode: Đạt tới lượng dữ liệu, dữ liệu tiếp theo được lưu lại từ vị trí đầu tiên của bộ đệm dữ liệu, giúp quá trình truyền nhận diễn ra liên tục và theo một vòng tròn.
  - Tự động tăng địa chỉ nguồn hoặc địa chỉ đích hoặc cả hai sau mỗi lần truyền/

Downloaded by Du Di?p (diepdu2005@gmail.com)

- Yêu cầu ngắt: Truyền dữ liệu đã thực hiện được một nửa quá trình hoặc đã hoàn thành việc truyền dữ liệu.



• Khai báo sử dụng DMA hoặc ngưng sử dụng DMA thông qua các hàm của thư viện HAL

```
ADC (stm32f4xx_hal_adc.c):
HAL_StatusTypeDef HAL_ADC_Start_DMA(ADC_HandleTypeDef* hadc, uint32_t* pData, uint32_t Length)
HAL_StatusTypeDef HAL_ADC_Stop_DMA(ADC_HandleTypeDef* hadc)
```

• Interrupts Service Routine được viết bên ngoài hàm main thông qua các hàm Callback của thư viện HAL

- Ưu điểm:
  - Nhanh hơn so với CPU.
  - Nâng cao hiệu suất.
  - Tự động tăng địa chỉ lưu trữ dữ liệu.
  - Tiết kiệm năng lượng.
- Nhược điểm:
  - Sử dụng không phù hợp sẽ tăng tiêu thụ năng lượng.

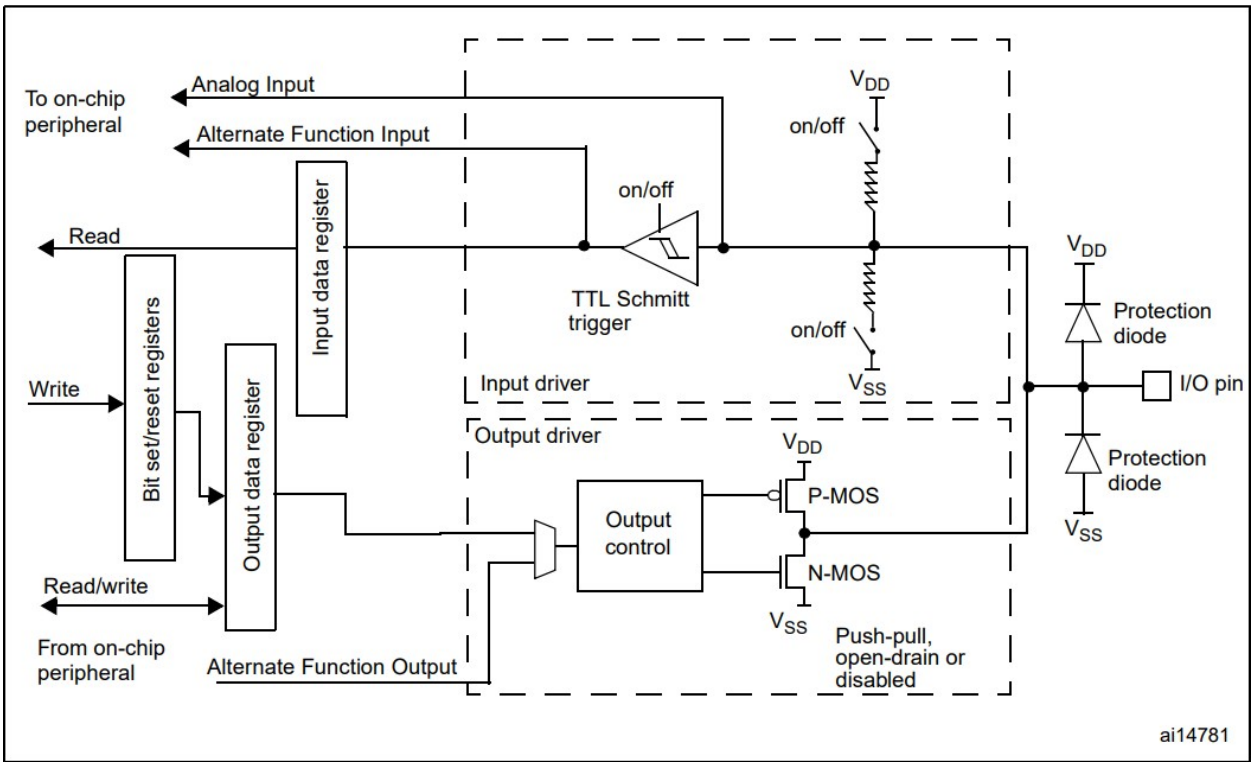
L. Lập trình ngoại vi GPIO

I. Cấu trúc và chức năng của khối GPIO

- Các chân I/O.
- Giao tiếp với thế giới bên ngoài.

Downloaded by Du Di?p (diepdu2005@gmail.com)

- Một khối GPIO gọi là 1 Port.
- Làm việc với các thanh ghi:
    - 2 thanh ghi cấu hình: GPIOx_CRL, GPIOx_CRH.
    - 2 thanh ghi dữ liệu: GPIOx_IDR, GPIOx_ODR.
    - Thanh ghi ghi dữ liệu trực tiếp tác động ở mức độ bit: GPIOx_BSRR.
        - Thanh ghi 16 bit: GPIOx_BRR.
    - Thanh ghi khóa cấu hình khối GPIO: GPIOx_LCKR.



- 4 tính năng:
    - Input
    - Analog
    - Output
    - Alternate Function: Kết nối ngoại vi khác
- Input: Bên ngoài tác động vào VĐK qua chân Input.
    - Khối TTL Schmitt Trigger xác định giá trị logic trên chân Input.
    - Thanh ghi Input Data chứa giá trị logic của chân Input.
    - Kết hợp với Pull up, Pull down
        - Input Floating: Không nối với trở, mức logic thay đổi ngẫu nhiên.
        - Input Pullup: Khi hở mạch, mức logic = 1

Downloaded by Du Di?p (diepdu2005@gmail.com)

        - Input Pulldown: Khi hở mạch, mức logic = 0
- Analog: Kết nối với các ngoại vi Analog như ADC, DAC
    - Ngoại vi Analog có thể đọc và tạo giá trị trên I/O Pin.
- Output: Điều khiển giá trị logic trên chân Output.
    - Giá trị ghi vào thanh ghi BSRR hoặc ODR.
    - Cấu hình khối Output Control điều khiển P-MOS và N-MOS theo các chế độ khác nhau.
    - Open–drain: Bit 0 – Logic 0, Bit 1– Trở kháng cao.
    - Push-pull: Bit 0 – Logic 0, Bit 1– Logic 1.
    - Bit tương ứng của thanh ghi ODR trong Port nào điều khiển Pin tương ứng.
- Alternate Function:
    - Có thể điều khiển, đọc giá trị Logic.

II. GPIO Output với LED

- GPIO output level: Mức logic đầu ra mặc định.
- Maximum output speed: Tốc độ chuyển trạng thái logic
- Lưu ý:
    - Giới hạn tối đa về dòng và áp.

Table 6. Voltage characteristics

Symbol	Ratings	Min	Max	Unit
$V_{DD} - V_{SS}$	External main supply voltage (including V_{DDA} and V_{DD})(1)	–0.3	4.0	V
V_{IN}(2)	Input voltage on 5 V tolerant pin	$V_{SS} - 0.3$	$V_{DD} + 4.0$	
	Input voltage on any other pin	$V_{SS} - 0.3$	4.0	
$\|\Delta V_{DDx}\|$	Variations between different V_{DD} power pins	-	50	mV
$\|V_{SSX} - V_{SS}\|$	Variations between all the different ground pins	-	50	
$V_{ESD(HBM)}$	Electrostatic discharge voltage (human body model)	See *Section 5.3.11*		

1. All main power (V_{DD}, V_{DDA}) and ground (V_{SS}, V_{SSA}) pins must always be connected to the external power supply, in the permitted range.
2. V_{IN} maximum must always be respected. Refer to *Table 7* for the maximum allowed injected current values.

Table 7. Current characteristics

Symbol	Ratings	Max.	Unit
I_{VDD}	Total current into V_{DD}/V_{DDA} power lines (source)(1)	150	mA
I_{VSS}	Total current out of V_{SS} ground lines (sink)(1)	150	
I_{IO}	Output current sunk by any I/O and control pin	25	
	Output current source by any I/Os and control pin	– 25	
$I_{INJ(PIN)}$(2)	Injected current on five volt tolerant pins(3)	-5/+0	
	Injected current on any other pin(4)	± 5	
$\Sigma I_{INJ(PIN)}$	Total injected current (sum of all I/O and control pins)(5)	± 25	

1. All main power (V_{DD}, V_{DDA}) and ground (V_{SS}, V_{SSA}) pins must always be connected to the external power supply, in the permitted range.

Downloaded by Du Di?p (diepdu2005@gmail.com)

    - Vấn dề tương thích giữa 2 thiết bị: Vin và Vout tương ứng.
        - CMOS: Giá trị điện áp tối thiểu của input high level có giá trị gần bằng 2/3 VDD, mức điện áp tối đa của input low level gần bằng 1/3 VDD.
        - TTL: Các giá trị tương ứng lần lượt là 2V và 0.8V.
    - Vấn đề năng lượng tiêu thụ của I/O Pin.
        - Tránh các chân không sử dụng ở chế độ floating, làm bộ TTL hoạt động liên tục. Nên thiết kế Pull up hoặc Pull down.
        - Điều chỉnh tốc độ đáp ứng của tín hiệu đầu ra. Slew rate có giá trị càng lớn thì khả năng sinh nhiễu trên GPIO Output càng nhiều.
    - Xem xét tắt Clock cho khối GPIO nếu không sử dụng trong một thời gian dài bằng hàm HAL_RCC_GPIOx_CLK_DISABLE().

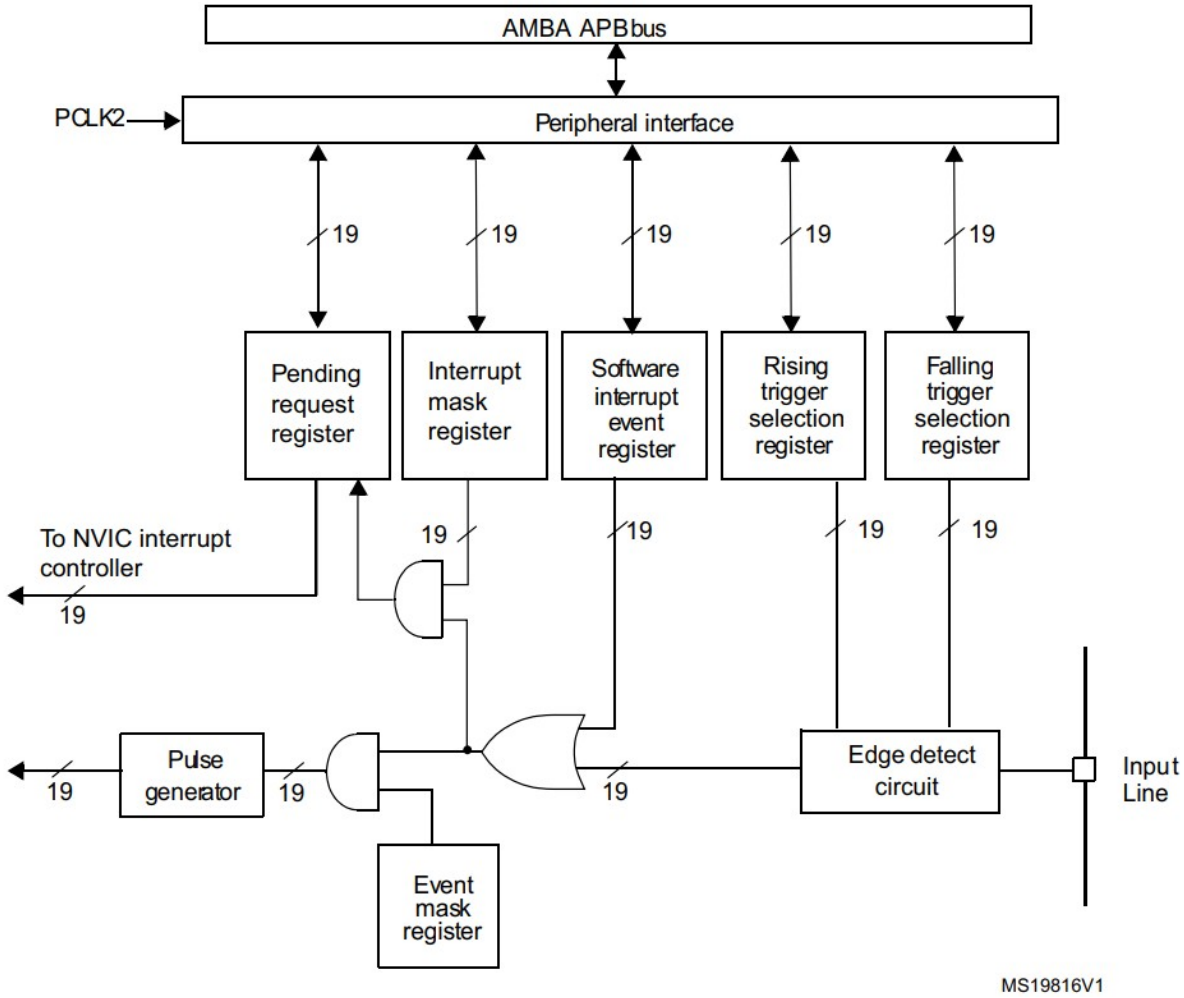
III. GPIO Input với Button

- Sử dụng Polling.
- Kinh nghiệm:
    - Đọc giá trị trên thanh ghi IDR để kiểm tra mức logic trên chân GPIO Input.
    - Kiểm tra thiết kế phần cứng với các cấu hình trên phần mềm.
    - Kiểm tra mức logic tương thích trên chân I/O của VĐK với chân I/O của ngoại vi.
    - Nhấn nút RD để cập nhật giá trị trên thanh ghi IDR.
- Vấn đề mở rộng:
    - Tolerant: Mức điện áp chấp nhận được của một chân I/O.
        - Three-Volt Tolerant (TC): Chấp nhận được điện áp tối đa = VDD + 0.3V.
        - Five-Volt Tolerant (FT): Chấp nhận được điện áp tối đa = VDD + 3.6V. (Chỉ phục vụ Input)
        - **Table 5- Datasheet**

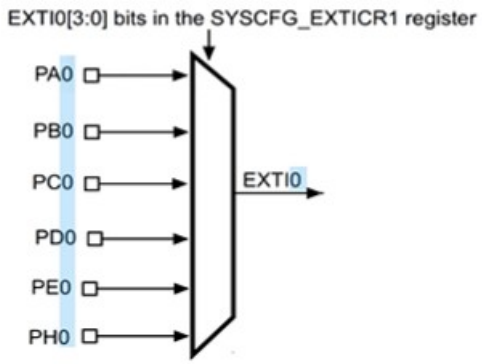
M. Lập trình ngắt ngoài EXTI.

I. Cấu trúc và chức năng của khối ngắt ngoài EXTI.

Downloaded by Du Di?p (diepdu2005@gmail.com)



- EXTI Input Line: là một tập hợp các chân có cùng thứ tự trên các Port khác nhau.
    - VD: EXTI0 gồm: PA0, PB0, …
    - Cấu hình lựa chọn tín hiệu nào từ chân được nối với Line 0 thông qua mạch chọn kênh (các bit)



Downloaded by Du Di?p (diepdu2005@gmail.com)

    - Một số nối với các I/O pin, một số nối với các ngoại vi.
- EXTI Input Line Signal: là tín hiệu điện, gồm 2 mức logic 0, 1, sườn lên, sườn xuống.
- Edge detect circuit: Mạch phát hiện sườn.
- Các thanh ghi: **10.3 – Reference Manual**
    - Thanh ghi cho phép ngắt: IMR
    - Thanh ghi chọn sườn lên: RTSR
    - Thanh ghi chọn sườn lên: FTSR
    - Nếu chọn cả sườn lên cả sườn xuống, ngắt thực hiện cả ở sườn lên và sườn xuống.
    - Thanh ghi chờ ngắt: PR: 1 cho tới khi ngắt được thực hiện.
- Call back, kiểm tra ngắt do cái gì:
    - GPIO_Pin == GPIO_PIN_0

II. EXTI với Button

- Các kiểm tra VXL không thực hiện chương trình phục vụ ngắt:
    - Xem lại nguyên lý phần cứng, cấu hình trở kéo, lựa chọn sườn ngắt, cấu hình cho phép ngắt và độ ưu tiên của ngắt.
    - Xem giá trị thanh ghi Trở kéo GPIOx_PUPDR, thanh ghi cho phép ngắt EXTI_IMR.
    - Kiểm tra thanh ghi pending EXTI_PR.
    - Nút Pause xem chương trình đang thực hiện câu lệnh nào.
- Lưu ý
    - Chương trình ngắt càng ngắn càng tốt.
    - Cẩn thận hàm HAL_Delay, nếu muốn thì phải set mức ưu tiên của ngắt cao hơn, nếu không sẽ bị treo ở lệnh HAL_Delay.
    - Lưu ý khai báo các biến toàn cục dùng chung giữa chương trình chính và chương trình ngắt với từ khóa volatile, để tránh CPU bỏ qua biến để tối ưu hiệu suất.
    - Với ngắt ngoài, không được để hở mạch ở chân sinh ra yêu cầu ngắt, nếu không chân sẽ thay đổi ngẫu nhiên mức logic, gây ra ngắt không mong muốn.

N. Giao tiếp UART

I. Cấu trúc và chức năng khối USART.

- Có 2 chế độ: Đồng bộ hoặc không đồng bộ -> UART.
- Serial

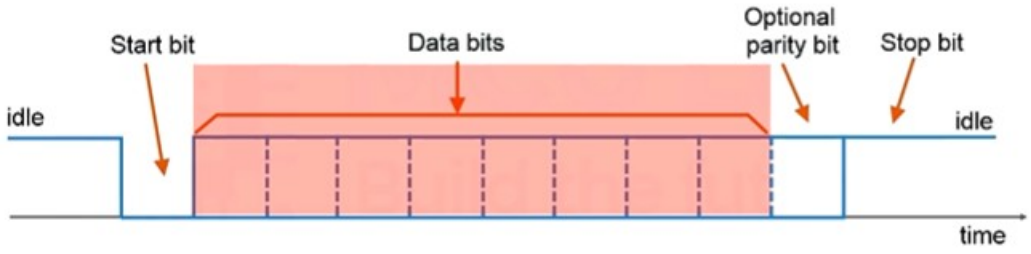
Downloaded by Du Di?p (diepdu2005@gmail.com)

1. **Kết nối phần cứng**
    - Tx nối với Rx và ngược lại.
    - Chung GND.

- Có thể có Hardware flow control.
    - CTS: Clear to send.
    - RTS: Request to send.
- Thiết kế cách ly quang nếu không chung GND.

2. **Khung truyền dữ liệu.**

- Trên đường truyền có các tín hiệu logic 0 và 1.
- Khi không truyền dữ liệu thì đường truyền sẽ có mức logic 1.
- Khi truyền thì mức logic sẽ thay đổi theo bit dữ liệu.



- Start bit: Bit bắt đầu với mức 0, báo hiệu truyền dữ liệu.
- Data bits: Bit dữ liệu, có thể 9 hoặc 8 bit, thường là 8, theo mã ASCII.
- Parity bit: Bit kiểm tra lỗi, có thể có hoặc không.
- Stop bit: Bit kết thúc, có thể 0,5/1/1,5/2 bit kết thúc, thường là 1.
- Hai bên cần thống nhất khung truyền.

3. **Tốc độ Baud**

- Không có tín hiệu Clock nối chung.
- Cần thống nhất tốc độ truyền dữ liệu giống nhau.
- Baudrate, đơn vị bps (bit per second, số bit truyền trong 1 giây).

II. Truyền dữ liệu nhiều định dạng qua UART.

- Sử dụng USB UART.
- Tìm chân thay thế: **Giữ Ctrl, Click chuột trái, chân tương ứng hiện đậm.**
- Truyền dữ liệu:

HAL_UART_Transmit…

    - *huart: Con trỏ trỏ tới Struct bộ UART đang sử dụng (Đang sử dụng USART6)

- HAL_UART_Transmit(&huart6, (uint8_t*)”H”, 1, 1000); Lệnh truyền kí tự

Downloaded by Du Di?p (diepdu2005@gmail.com)

```c
MX_GPIO_Init();
MX_USART6_UART_Init();
/* USER CODE BEGIN 2 */
HAL_UART_Transmit(&huart6, (uint8_t*)"H", 1, 1000);
HAL_UART_Transmit(&huart6, (uint8_t*)"\n", 1, 1000);
__NOP();

HAL_UART_Transmit(&huart6, (uint8_t*)"MCULearning", 11, 1000);
HAL_UART_Transmit(&huart6, (uint8_t*)"\n", 1, 1000);
__NOP();

uint16_t intnumber = 1000;
char intarray[5] = {0};
sprintf(intarray,"%d",intnumber);
HAL_UART_Transmit(&huart6, (uint8_t*)intarray, 4, 1000);
HAL_UART_Transmit(&huart6, (uint8_t*)"\n", 1, 1000);
__NOP();

float floatnumber = 24.8;
char floatarray[5] = {0};
sprintf(floatarray,"%.1f",floatnumber);
HAL_UART_Transmit(&huart6, (uint8_t*)floatarray, 4, 1000);
HAL_UART_Transmit(&huart6, (uint8_t*)"\n", 1, 1000);
__NOP();

uint8_t counter = 0;
char counterarray[15] = {0};

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    sprintf(counterarray,"Counter: %03d\r\n",counter);
    HAL_UART_Transmit(&huart6, (uint8_t*)counterarray, strlen(counterarray), 1000);
    counter++;
    HAL_Delay(1000);
  /* USER CODE END WHILE */
```

'H'

"MCULearning"

"1000"

"24.8"

"Counter: 000"

- Kí tự kết thúc câu: \r
- Nếu muốn hiển thị chuỗi kí tự ở dạng HEX:

```c
char hexarray[6] = {0};
sprintf(hexarray,"%c%c%c\r\n",0x1,0x2,'A');
HAL_UART_Transmit(&huart6, (uint8_t*)hexarray, 5, 1000);
```

III. Nhận dữ liệu UART sử dụng ngắt nhận RXNEIE.

- Khai báo mã bộ đệm chứa dữ liệu:

Char Rx_buffer[13]={0};

- Hàm nhận dữ liệu

HAL_UART_Receive_IT(&huart6, (uint_t*)Rx_buffer, 13);

- Trong hàm Callback, truyền ngược lại và xóa dữ liệu trong bộ đệm nhận dữ liệu.

For (uint_t i=0; i<13; i++)
Rx_bufer[i] = 0;

Downloaded by Du Di?p (diepdu2005@gmail.com)

- Phương pháp nhận biết dữ liệu đã được gửi xong.
    - Dựa vào kí hiệu kết thúc (\r hoặc \n).
    - Dựa vào thời gian từ thời gian nhận byte dữ liệu cuối cùng đến thời gian hiện tại, nếu vượt quá một khoảng thời gian nhất định nào đó thì có nghĩa VXL đã nhận xong dữ liệu. Khoảng thời gian đó dựa trên baudrate để tính toán hợp lý.
- Khai báo một bộ đếm chứa ký tự đếm:

```
Char Rx_byte[2] = {0};
```

- Khai báo một biến đếm:

```
uint8_t Rx_counter = 0;
```

- Nhận từng kí tự:

```
HAL_UART_Receive_IT(&huart6, (uint_t*)Rx_byte, 1);
```

- Trong hàm Callback:

```
Rx_buffer[Rx_counter] = Rx_byte[0];

Rx_counter = Rx_counter + 1;
```

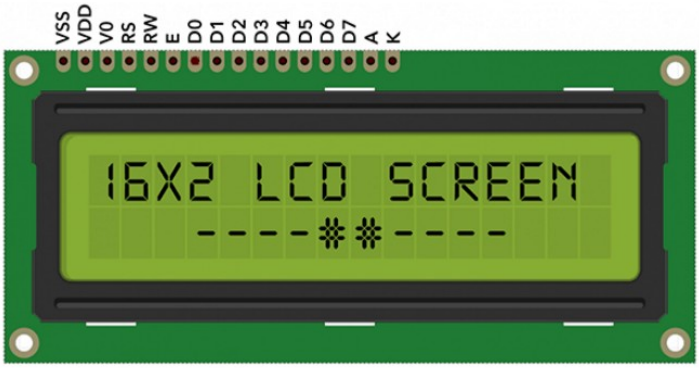
- Kiểm tra có phải kí tự kết thúc ko, xóa bộ đệm.
- Lưu ý:
    - Kiểm tra thanh ghi liên quan:
        - Cờ báo lỗi: USART_SR.
        - Data Register: USART_DR.
        - Cấu hình cho phép truyền hoặc nhận dữ liệu trong thanh ghi: USART_CR1.
    - Kiểm tra phần cứng kết nối.
    - Kiểm tra baudrate.
    - Kiểm tra NVIC.
    - Cẩn thận hàm HAL_Delay

M. Ứng dụng thực tế

I. LCD 1602

1. Sơ đồ chân

Downloaded by Du Di?p (diepdu2005@gmail.com)



No	Symbol	Function
1	VSS	Ground
2	VDD	5V +
3	V0	Contrast
4	RS	Register
5	RW	Read/Write
6	E	Enable
7	D0	Data bus
8	D1	Data bus
9	D2	Data bus
10	D3	Data bus
11	D4	Data bus
12	D5	Data bus
13	D6	Data bus
14	D7	Data bus
15	A	Anode (5V+)
16	K	Cathode (GND)

- VDD: chân nguồn, nối với 3,3V.
- VSS: Nối đất
- V0: chân độ tương phản, thường nối với biến trở để điều chỉnh độ tương phản.
- RS: Chân chọn thanh ghi.
    - Cho phép ghi dữ liệu vào thanh ghi lệnh và dữ liệu.
    - Ghi vào thanh ghi lệnh: Cho phép ghi lệnh Reset, Clear, Lệnh điều khiển con trỏ.
    - Ghi vào thanh ghi dữ liệu: Cho phép ghi dữ liệu lên màn hình.
- RW: Chân đọc/viết, thường chỉ sử dụng Write, đọc chỉ để xem LCD có bận hay không.
- E: Enable, nối vào VDD để cho phép hoạt động.
- Dx: 8 chân IO.
    - Có thể sử dụng moduln I2C để truyền dữ liệu trung gian.
    - Có thể sử dụng trực tiếp GPIO, kết nối trực tiếp với VĐK.
- A, K: Cấp nguồn cho đèn nền.

2. Sử dụng thư viện

- Sử dụng thư viện lcd16x2 để thực hiện giao tiếp giữa LCD và STM32.
- Cần thêm 2 file .c và .h vào Core -> Inc và Src.
- File .h: Định nghĩa các hằng số, cấu trúc và khai báo hàm sẽ sử dụng.

```
// lcd.h

#ifndef LCD_H

#define LCD_H


// Các hằng số và khai báo hàm

void LCD_Init(void);
```


Downloaded by Du Di?p (diepdu2005@gmail.com)

```
   void LCD_Clear(void);
   void LCD_SetCursor(uint8_t row, uint8_t col);
   void LCD_Print(char *str);

   #endif
```

- File .c: Viết mã nguồn thực tế dựa trên các khai báo từ File .h.

```
// lcd.c
#include "lcd.h"

// Định nghĩa các hàm được khai báo trong lcd.h
void LCD_Init(void) {
    // Khởi tạo LCD
}

void LCD_Clear(void) {
    // Xóa màn hình LCD
}

void LCD_SetCursor(uint8_t row, uint8_t col) {
    // Đặt con trỏ vị trí trên LCD
}

void LCD_Print(char *str) {
    // In chuỗi lên LCD
}
```

3. Code

- Include thư viện lcd_16x2.h.
- Khai báo các chân sử dụng trong file lcd_16x2.h.
- Khởi tạo Lcd_Init();
    - Cấu hình các chân trong LCD.

Downloaded by Du Di?p (diepdu2005@gmail.com)

- GPIO_InitTypeDef GPIO_InitStruct = {0}; Khai báo biến 'GPIO_InitStruct' để sử dụng cho cấu hình GPIO. Biến này sẽ chứa các thông số cấu hình như chế độ, resistor pull-up/pull-down, tốc độ, …
- // GPIO clock for control and data lines
-
- __HAL_RCC_GPIOA_CLK_ENABLE(); Bật Clock tại các cổng
- __HAL_RCC_GPIOB_CLK_ENABLE();
- __HAL_RCC_GPIOC_CLK_ENABLE();
-
- // Configure RS_PIN
- GPIO_InitStruct.Pin = RS_Pin; Khai báo chân muốn cấu hình.
- GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP; Cấu hình chế độ cho chân GPIO: Chế độ đầu ra Push-Pull, có thể điều khiển tín hiệu điện áp cao hoặc thấp.
- GPIO_InitStruct.Pull = GPIO_NOPULL; Cấu hình Resistor pull-up/pull-down trên chân GPIO, mức logic ban đầu cho chân.
- GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH; Cấu hình tốc độ chuyển đổi cho chân GPIO.
- HAL_GPIO_Init(RS_GPIO_Port, &GPIO_InitStruct); Khởi tạo các thông số cài đặt cho chân GPIO vừa thiết lập.

- Thư viện LCD có các hàm:
  - Xóa màn hình.
  - Viết 1 biến.
  - Viết 1 string.
  - Tạo một kí tự người dùng.
  - Đi đến vị trí x, y: Tại cột, hàng nào.
- Ví dụ

```
/* USER CODE BEGIN 2 */

  Lcd_gotoxy(5,0);

  Lcd_write_string("STM32");

  Lcd_gotoxy(3,1);

  Lcd_write_string("BlackPink");

 /* USER CODE END 2 */


 /* Infinite loop */

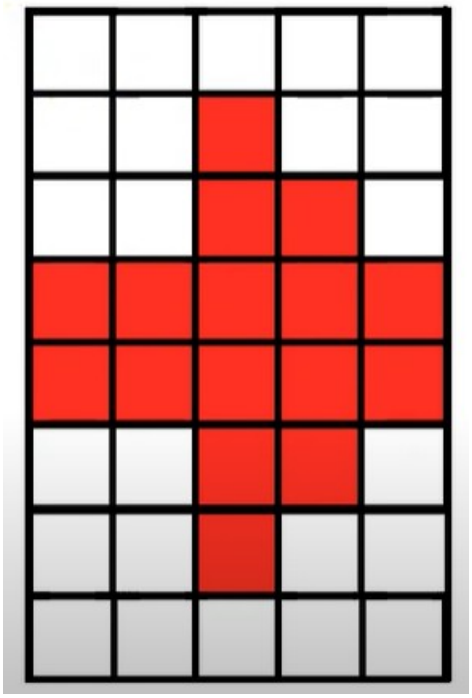
 /* USER CODE BEGIN WHILE */
```


Downloaded by Du Di?p (diepdu2005@gmail.com)

```
while (1)
{
  /* USER CODE END WHILE */
      HAL_Delay(3000);
      Lcd_clear_display();
  /* USER CODE BEGIN 3 */
}
```

3. Tạo kí tự mũi tên

- 1 ô gồm 5 cột và 8 hàng.
- Tạo các mã hexa tương ứng.



  - uint8_t Right_char[8]={0x00, 0x04, 0x06, 0x1F, 0x1F, 0x06, 0x04, 0x00};
  - uint8_t Left_char[8]={0x00, 0x04, 0x0C, 0x1F, 0x1F, 0x0C, 0x04, 0x00};
- Lcd_creat_custom_char(0, Right_char); 0 là địa chỉ trong DRAM.
- Lcd_write_custom_char(10, 1, 0);

II. PWM

- Công thức tính tần số hoạt động:

Downloaded by Du Di?p (diepdu2005@gmail.com)

$$F_t = \frac{F_s}{(Prescale+1)(Period+1)}$$

- Pulse: Độ rộng xung
- Code:
    - HAL_TIM_PWM_Start(TIM_HandleTypeDef *htim, uint32_t Channel): Khởi động PWM.
        - HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
    - __HAL_TIM_GET_COMPARE((&htim1, TIM_CHANNEL_1, var); Thay đổi Pulse = var.

III. Encoder

- Code:
- HAL_TIM_Encoder_Start(&htim1, TIM_CHANNEL_1| TIM_CHANNEL_2); Khởi động Encoder.
- __HAL_TIM_GET_Counter(&htim1): Lấy giá trị từ thanh ghi bộ đếm.

IV. ADC

- Chỉ đọc được 12 bit dữ liệu.
- Có thể chỉnh tốc độ lấy mẫu.
- Bắt đầu ADC:
    - HAL_ADC_Start(&hadc1);
- Đọc giá trị ADC:
    - HAL_ADC_GetValue(&hadc1);
- Chuyển đổi liên tục ADC:
- Kiểm tra điều kiện bộ ADC nào được hoàn thành (nhảy vào ngắt):
    - hadc -> Instance == hadc1.Instance
- ADC với DMA

V. Bộ nhớ Flash.

- 128 KB chia thành 128 page, mỗi page có 1 địa chỉ riêng gồm 400 thanh ghi.
- Sử dụng công nghệ cổng NAND, ghi đọc theo từng page, xóa dữ liệu trên cả page để ghi đè lên.

Downloaded by Du Di?p (diepdu2005@gmail.com)